Índice de contenido

1. Introducción a CSS (Hojas de estilo en cascada)....2
2. Crear y vincular hojas de estilo....3
3. Selectores : Etiquetas, clases, identificadores y otros....5
4. Unidades de medida. Modelo de cajas....10
5. Posicionamiento y visualización....22
6. CSS Grid....35
7. Colores, tipografía, texto, fondo, listas....36
8. Avanzado : botones, degradados, menús, tablas, transformaciones, etc....48
9. Flexbox....58
10. Uso de propiedades personalizadas CSS (variables)....67

1. Introducción a CSS (Hojas de estilo en cascada)

La filosofía de CSS (Cascading Style Sheet) es la siguiente : En el <body> del documento HTML se define el contenido que se muestra en el sitio y luego en otro archivo (o en el <head>) se define la apariencia de la página usando CSS. Esto permite que se pueda modificar el contenido del <body> sin cambiar su apariencia y también modificar su apariencia sin modificar el contenido.

Basaremos esta unidad didáctica en los documentos :

01IntroduccionCSS.pdf
02Introduccion_CSSLibrosWeb.pdf
03CSS_avanzadoLibrosWeb.pdf
04CursoCSS3.pdf

Para ampliar información sobre la estructura de la página html puedes ver :

http://es.learnlayout.com/toc.html

2. Crear y vincular hojas de estilo

Para asociar código HTML con estilo CSS podemos :

1) Usar el atributo style dentro de las etiquetas HTML.

 El fondo de este párrafo es azul

Presenta alta prioridad frente a otras reglas. Dificulta el mantenimiento. Volver a escribir los atributos para otro elemento igual.

2) Escribir la etiqueta <style> en el <head> del documento. Ejemplo :

```
<head>
        <style type="text/css" media="screen, tv" >
                -----
        </style>
</head>
```

media indica sobre que dispositivo se aplica la regla. Algunos valores : **print** (para impresoras) , **screen** (para pantallas), **tv** (televisores), **all** (para todos). Si son varios van separados por comas.

3) En un archivo externo :

```
<link href="style.css" rel="stylesheet">
```

O utilizando hojas de estilo alternativas. En el menú **ver** se puede ver la página con cada uno de sus estilos.

```
<link href="default.css" rel="stylesheet" title="Estilo por defecto">
<link href="lujo.css" rel="alternate stylesheet" title="Lujoso">
<link href="basic9.css" rel="alternate stylesheet" title="Basico">
```

Podemos controlar si la hoja de estilos se cargó correctamente o se produjo algún error (Esta parte requiere conocimientos de javascript que se ven en otro módulo).

```
<script>
function sheetLoaded() {
  // Hacer algo interesante; la hoja de estilos ha sido cargada
}

function sheetError() {
  alert("¡Ocurrió un error al cargar la hoja de estilos!");
}
</script>
```

```
<link rel="stylesheet" href="mystylesheet.css" onload="sheetLoaded()"
onerror="sheetError()">
```

Este último modo (el modo 3) es el que utilizaremos de forma general aunque en algunos ejemplos y por motivos pedagógicos utilizaremos los métodos 1 y 2.

Se puede ampliar esta información en : **03CSS_avanzadoLibrosWeb.pdf** en el apartado 6.4 Estilos alternativos, páginas 142 a 144.

Ejercicio : Prueba a cargar varias hojas de estilo alternativas (como en el apartado 3) y comprueba que funcionan en tu navegador (con poner body { backgroung-color: elquesea ; } en cada uno de los archivos css ya podrás comprobar si funciona)

Ejercicio: Prueba a crear hojas de estilo alternativas, para screen y para print (altera el color de algo, por ejemplo) y pruebalo (la opción de imprimir puedes hacerlo sobre un archivo, no hace falta que sea en papel).

Este último ejercicio, tal y como está explicado en la unidad, funciona para IE pero no para los otros navegadores. Se puede realizar también :

1) Haz un link externo como en el punto 3 anterior.

2) Incluye dentro del enlace :

@media print {

...

- css para el caso de la impresora -

...

}

3. Selectores : Etiquetas, clases, identificadores y otros

El selector es la parte de la regla CSS que indican a qué elemento o parte de una página se aplica un determinado estilo.

Etiquetas. Se utilizan las propias etiquetas HTML como selectores.

Selector { atributo : valor }

```
<head>
<title> Ejemplo </title>
<style>
        h1, h2 { color : red; background-color: blue }  ==> Con la "," agrupamos dos etiquetas
        p { color: green }
</style>
</head>
```

Clases. Las etiquetas HTML pueden pertenecer a una clase o varias. Ejemplo :

```
<body>
<h1 class="alerta"> Texto de h1 </h1>
<h2 class="alerta"> Texto de h1 </h1>
<p class="alerta parrafo" > Texto de p con dos clases </p>
</body>
```

En CSS :
.alerta { color: red; }

H1.alerta { color: red; } ==> Hace referencia a los H1 de clase "alerta".

Observa el punto antes de la palabra alerta.

Id. El id de un documento html debe ser único. Ejemplo : ….

<h1 id="principal" class="alerta">Perigo!</h1> Coidado...	h1.alerta {color: red;} ==> Un h1 de clase alerta .alerta {color:red;} ==> La clase alerta #principal { font-size:50px; } == El id principal

#unico {color : blue; } ==> Nos referimos a la etiqueta con id = "unico"

Descendientes. Un elemento es descendiente de otro cuando se encuentra entre las etiquetas de apertura y cierre del otro elemento.

<ul> <li>Texto 1</li> <li>Texto 2</li> </ul> Texto texto texto3 Texto	ul b {color: purple;} ==> Selecciona Texto1 pero no Texto3. Selecciona a los **b** que tienen un ancestro **ul**

En el siguiente ejemplo :

p a span { color: red; } ==> Se aplica a los elementos **span** que tienen un ancestro **a** y que está dentro de un **p**. Observa que hay espacios en blanco entre las etiquetas.

Parentesco. Se trata de seleccionar hijos directos de otros elementos. Se indica con el signo "mayor que" (>) .

<ul class="menu"> <li>Arquivo <ul> <li>Abrir</li> <li>Gardar</li> <li>Gardar como...</li> </ul> </li> <li>Edicion <ul> <li>Copiar</li> <li>Mover</li> <li>Pegar</li> </ul> </li> </ul>	ul.menu li { color: blue; } ==> Selecciona todos los li del ejemplo . Ya que todos tiene un ancestro ul de la clase "menu" ul.menu > li {color: white;} ==> Selecciona los li Arquivo y Edición. Los otros no al no ser hijos directos.

P > span { color : red ; } ==> Se podría leer como los span que son hijos directos de p .

Adyacente. Sintaxis : Elemento1 + Elemento2 { color: red; } ==> Observa signo mas.
Selecciona los que cumplan las siguientes condiciones :

- Elemento1 y elemento2 deben ser hermanos, es decir, hijos del mismo padre
- Elemento2 debe aparecer inmediatamente despues de elemento1 en la hoja html.

<body> <h1>Titulo1</h1> <h2>Subtítulo</h2> ... <h2>Otro subtítulo</h2> ... </body>	h1 + h2 { color: red } Selecciona el primer h2 pero no el segundo.

Hermanos. El combinador ~ separa dos selectores y selecciona el segundo elemento sólo si está precedido por el primero y ambos comparten un padre común.

<section>
Este span no es rojo.
 Aquí hay un párrafo.
<code>Aquí hay algo de código.</code>
Aquí hay un span. Es rojo
</section>
	p ~ span {
	color: red;
	}

p ~ span ==> Selecciona los span que son hermanos de p y que están a continuación de p

Atributos. Podemos seleccionar un elemento por su atributo.

<a href="#">Internal link</a> <a title="Ir a marca " href="marca.com"> Marca </a>	a[title] { color:red; } ==> Selecciona todos los <a> que tienen el atributo title a[href="#"] { color:black; } ==> Selecciona todos los a que tienen # en href

Podemos ampliar información sobre este modo de selección en :
02Introducción_cssLibrosWeb.pdf página 27.

Pseudo-clases (Se usan para seleccionar estados de un elemento) :

:link ==> Se aplica a un link (<a>) que no ha sido visitado.
:visited ==> Se aplica a un link (<a>) visitado
:hover ==> Cuando el ratón pasa por encima de un elemento (no tiene porqué ser <a>)
:active ==> Cuando el elemento se activa al pulsar click, por ejemplo.
:focus ==> En caso de formulario, cuando el elemento tiene el foco (el cursor).

:nth-child(n) ==> Selecciona el hijo n del elemento. Admite expresiones complejas como :

li:nth-child(2n+1) ==> Selecciona los li impares
p:nth-child(2n) ==> Selecciona los p pares

p:nth-child(4n+1) ==> Alternan 4 estilos diferentes para los párrafos.
p:nth-child(4n+2)
p:nth-child(4n+3)
p:nth-child(4n+4)

Mas información de nth-child en :
https://developer.mozilla.org/es/docs/Web/CSS/:nth-child

Mas pseudo-clases en :
https://www.w3schools.com/css/css_pseudo_elements.asp

Pseudo-elementos (Se usan para seleccionar partes de un elemento) :

::first-line ==> Primera linea de un elemento
::first-letter ==> Primera letra de un elemento
::before ==> Para insertar algún contenido antes del contenido de un elemento
::after ==> Para insertar algún contenido después del contenido de un elemento
::selection ==> La selección que un usuario hace con el ratón de un elemento o parte.
(Se utiliza content con ::before y ::after)

<h1>Hola Manola</h1>	h1::before { content: url(smiley.gif); } ==> Se ve una carita antes del texto “Hola Manola”

<h1>Hola Manola</h1>	h1::after { content: url(smiley.gif); } ==> Se ve una carita después del texto “Hola Manola”

<h1>Algún texto de tipo h1</h1> Esto es un párrafo Esto es un texto en un div	/*Para Mozilla hay que poner lo siguiente : */ ::-moz-selection { color: red; background: blue; } ::selection { color: red; background: yellow;} Cuando se selecciona con el ratón algún texto se verá rojo con fondo azul.

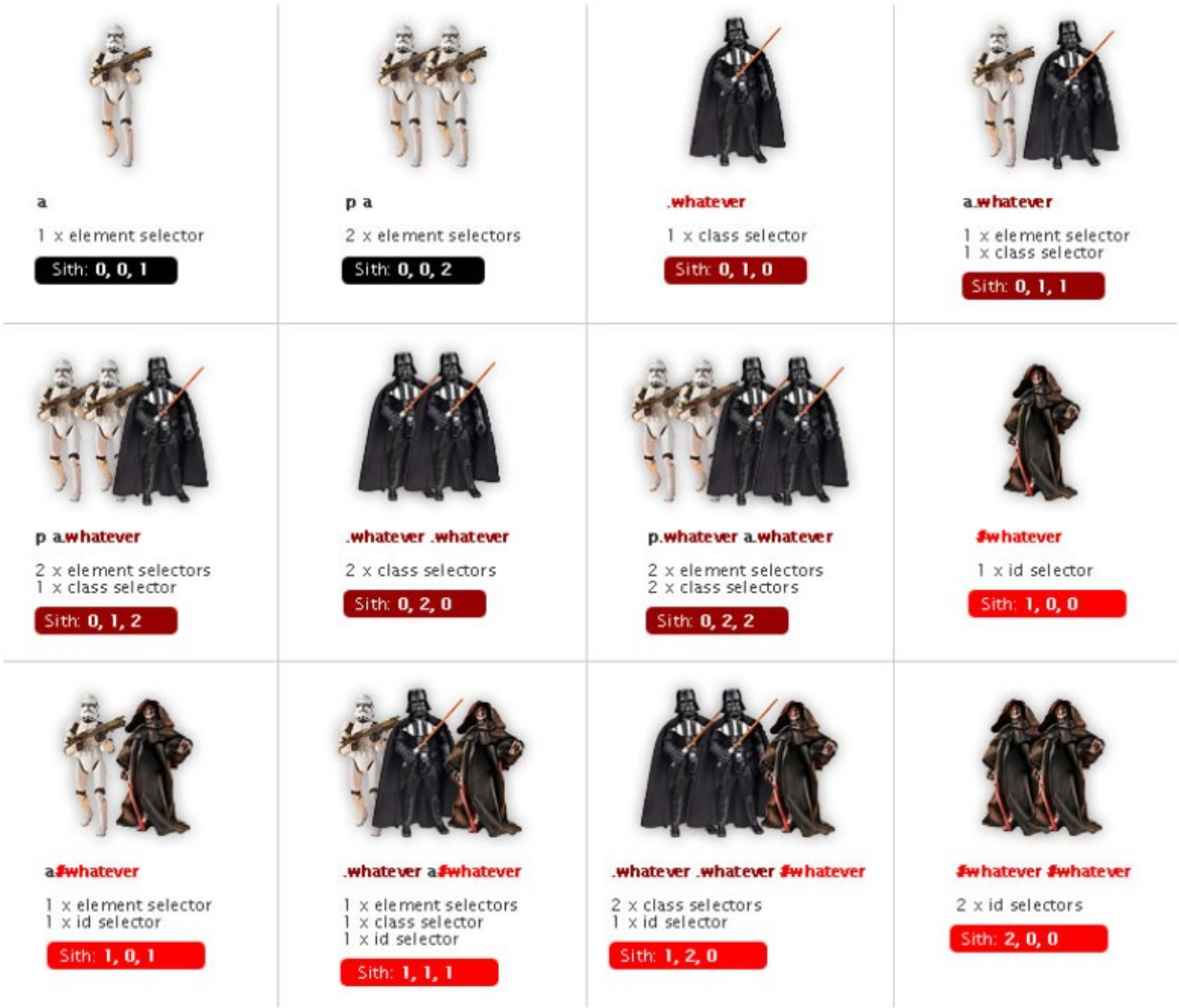
Los :: se usan en CSS3 para diferencian entre pseudo-elementos y pseudo-clases.
Pseudo-elementos ==> Estilo a parte de un elemento (ej : primera letra , primera línea)
Pseudo-clases ==> Estilo del elemento

Hay veces que un mismo elemento tiene distintas reglas CSS, por ejemplo :

p { color: blue; }

.alerta p { color: red; }

Básicamente diremos que cuanto mas específica es una regla, mayor prioridad tiene. Para ver este punto en detalle consultar : página 22 de 01Introduccion_CSS.pdf donde además ofrece el siguiente gráfico :



Cuanto mayor es el número, mas prioridad tiene la regla CSS.

4. Unidades de medida. Modelo de cajas

https://www.freecodecamp.org/espanol/news/aprende-unidades-de-medidad-css-em-rem-vh-y-vw-con-ejemplos-de-codigos/

Unidades absolutas
Son fijas y **no dependen del entorno** (pantalla, navegador, dispositivo).
👉 No son recomendables para diseño web, salvo en impresión (`print`).

- **px (píxeles)** → unidad más usada en pantalla.

```
p {
  font-size: 16px;   /* texto fijo en 16 píxeles */
}
```

- Otras unidades:
  - **pt (puntos)** → 1/72 de pulgada.
  - **cm (centímetros)**, **mm (milímetros)**, **in (pulgadas)**.
  - **pc (picas)** → 1pc = 12pt.

⚠ Usarlas en web no es buena práctica porque no escalan en distintos dispositivos.

Unidades relativas a la tipografía

Se adaptan al tamaño de la fuente, lo que las hace más flexibles.

- **em** → relativo al tamaño de fuente del elemento **padre**.

```
div {
  font-size: 20px;
}
p {
  font-size: 2em;   /* equivale a 40px porque el padre vale 20px */
}
```

- **rem (root em)** → relativo al tamaño de fuente del elemento **raíz** (`html`).

```
html {
  font-size: 16px;
}
h1 {
  font-size: 2rem;  /* 32px siempre, independientemente del padre */
```

```
}
```

👉 `rem` suele usarse en tipografía porque da consistencia en todo el documento.

- **ex** → relativo a la altura de la letra "x" de la fuente usada.
- **ch** → relativo al ancho del carácter "0" de la fuente usada.

```
p {
  width: 40ch;   /* el ancho del párrafo será como 40 caracteres "0" */
}
```

Unidades relativas al viewport (ventana gráfica)

Muy importantes para diseños responsive.

- **vh (viewport height)** → 1vh = 1% de la altura de la ventana.
- **vw (viewport width)** → 1vw = 1% del ancho de la ventana.
- **vmin** → 1% del lado menor (alto o ancho, el más pequeño).
- **vmax** → 1% del lado mayor (alto o ancho, el más grande).

Ejemplo:

```
div {
  width: 50vw;   /* 50% del ancho de la ventana */
  height: 50vh;  /* 50% de la altura de la ventana */
}

h1 {
  font-size: 5vmin;  /* el texto será grande, adaptado al lado más pequeño */
}
```

🔎 **¿Qué es `vmin`?**

- `1vmin` = **1% del lado más pequeño del viewport** (pantalla visible).
- El "lado más pequeño" puede ser **el ancho** o **el alto**, dependiendo de cuál sea menor en ese momento.

👉 Entonces:

- En un monitor apaisado (horizontal), **la altura suele ser menor que el ancho**, por lo que `vmin` toma como referencia la altura.

- En un móvil vertical, **el ancho suele ser menor que la altura**, por lo que `vmin` toma como referencia el ancho.

📐 **Ejemplo práctico**
Supongamos que tenemos un navegador de **1200px de ancho × 800px de alto**:

- El lado menor = `800px`.
- Entonces:
  - `1vmin = 8px` (1% de 800).
  - `5vmin = 40px`.

Si ponemos:

```
h1 {
  font-size: 5vmin;
}
```

El texto se verá a **40px** de tamaño.

Ahora imagina girar el dispositivo: **800px de ancho × 1200px de alto**:

- El lado menor = `800px` (¡ahora el ancho es el más pequeño!).
- `1vmin = 8px` → `5vmin = 40px` otra vez.

El tamaño de letra cambia de forma **proporcional al tamaño más limitado de la pantalla**, asegurando que el texto nunca se desborde ni quede enorme en pantallas estrechas.

Porcentajes
El valor depende siempre del **contenedor padre**.

```
div {
  width: 80%;   /* 80% del ancho del padre */
}
```

👉 Muy usados para layouts fluidos.

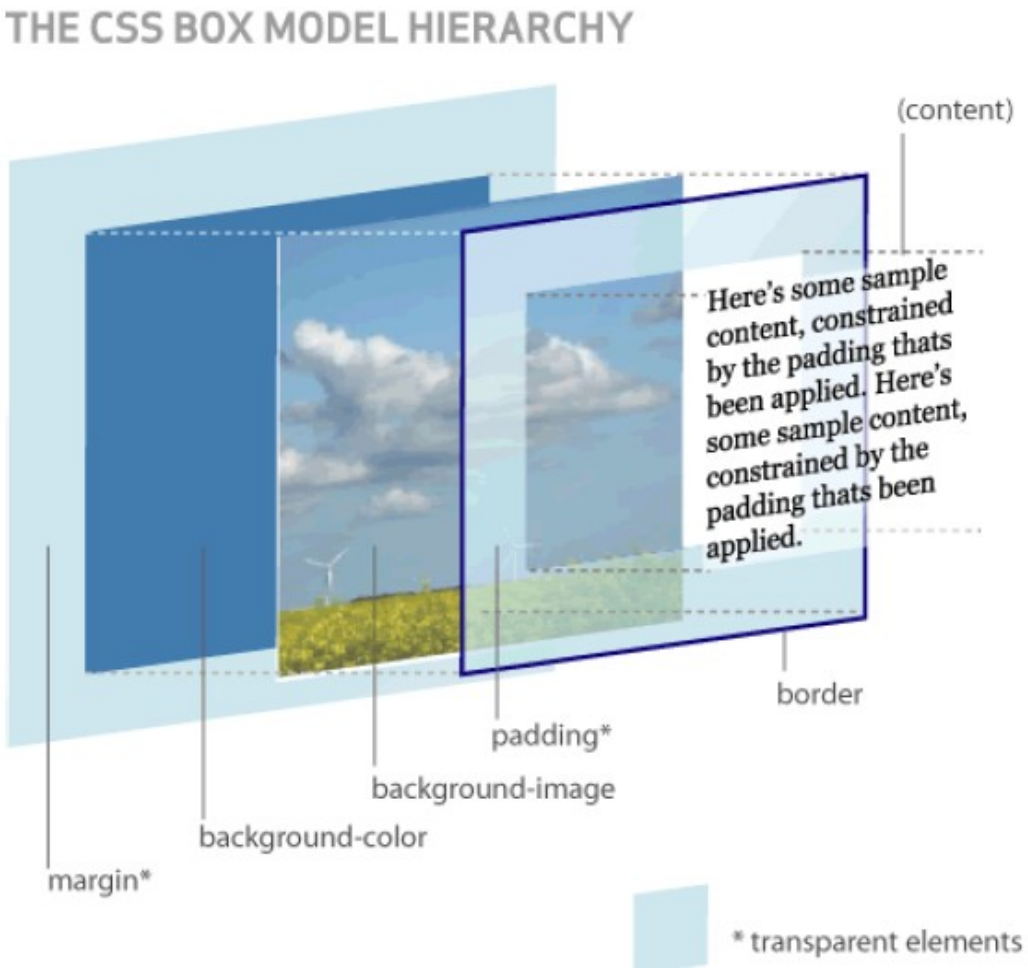
Función `calc()`

Permite **combinar unidades** en operaciones matemáticas.

- Suma, resta, multiplicación y división:

```
div {
  width: calc(100% - 50px);   /* 100% del padre menos 50px */
}

p {
  font-size: calc(1rem + 1vw);  /* fuente base que escala con pantalla */
}
```



Para entender los estilos CSS lo más adecuado es pensar en que cada elemento (<p> <h1> etc) es una caja. Con esta definición una página es una caja de cajas.

El modelo de cajas

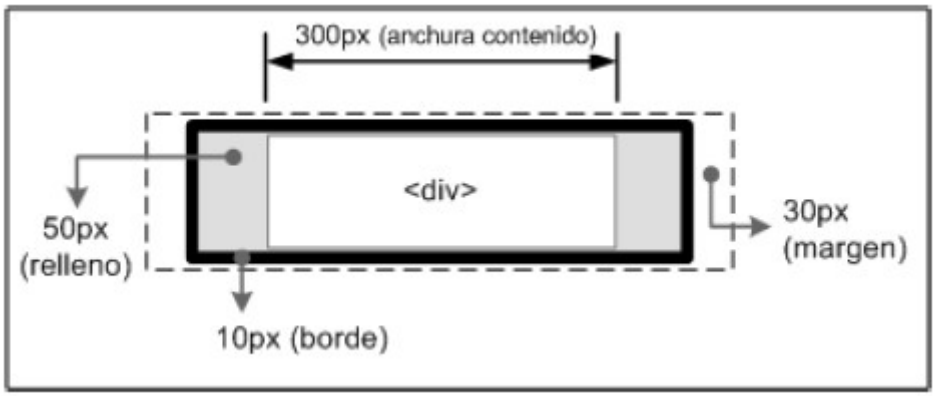
- El tamaño del **contenido** se define con:
  - **width**
  - **height**
    - <medida>
    - <porcentaje>
    - auto: lo ajusta el navegador
    - inherit: valor del elemento padre
- Propiedades del **borde**
  - Estilo del borde: **border-style**
    - none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset | inherit
  - Anchura: **border-width**
  - Color: **border-color**
  - Lados individuales: **border-top-style, border-right-style, border-bottom-style, border-left-style**
  - Resumen: **border**
    - (<medida_borde> || <color_borde> || <estilo_borde>) | inherit
  - CSS3 añade bordes redondeados y sombras

Importantes :
min-width: 500px; ==> Establece un valor mínimo de la caja.
max-width : 500px; ==> Establece un valor máximo de la caja.
min-height : 300px; ==> Idem con la altura.
max-height : 500px; ==> Idem con la altura.

El modelo de cajas

- Fondo
  - Un color: **background-color**
    - <color> | transparent | inherit
  - Una imagen: **background-image**
    - <url> | none | inherit
    - La imagen se puede repetir, esto se controla con **background-repeat**
      - repeat | repeat-x | repeat-y | no-repeat | inherit
    - Qué ocurre con la imagen de fondo cuando se hace scrolling en la página: **background-attachment**
      - scroll | fixed | inherit
  - Todas las propiedades: **background**
    - (<background-color> || <background-image> || <background-repeat> || <background-attachment> || <background-position>) | inherit

Anchura de una caja : contenido + padding (relleno) + border-bottom



Propiedad box-sizing : border-box ;

Por defecto, cuando defines `width` y `height` en CSS, estos **solo afectan al contenido**, y luego se añaden el padding y el borde.

- **`content-box` (valor por defecto):**
  El `width` y `height` **solo incluyen el contenido**.

```
div {
  width: 200px;       /* contenido = 200px */
  padding: 20px;      /* total = 240px */
  border: 10px solid;
  box-sizing: content-box;
}
```

- **`border-box`:**
  El `width` y `height` **incluyen contenido + padding + border**.
  👉 Muy usado porque facilita el diseño.

```
div {
  width: 200px;       /* incluye contenido + padding + border */
  padding: 20px;
  border: 10px solid;
  box-sizing: border-box;
}
```

- Con `content-box`, el tamaño final puede ser mucho mayor de lo esperado.

- Con `border-box`, el tamaño siempre es el declarado.

Ver :

https://developer.mozilla.org/es/docs/Web/CSS/box-sizing

Border-style . Ejemplo de valores.

Este es un párrafo con el borde en estilo none

Este es un párrafo con el borde en estilo hidden, aparentemente es igual que el estilo none, pero se diferencia en el trato que se le da a las tablas.

Este es un párrafo con el borde en estilo dotted

Este es un párrafo con el borde en estilo dashed

Este es un párrafo con el borde en estilo solid

Este es un párrafo con el borde en estilo double

Este es un párrafo con el borde en estilo groove

Este es un párrafo con el borde en estilo ridge

Este es un párrafo con el borde en estilo inset

Este es un párrafo con el borde en estilo outset

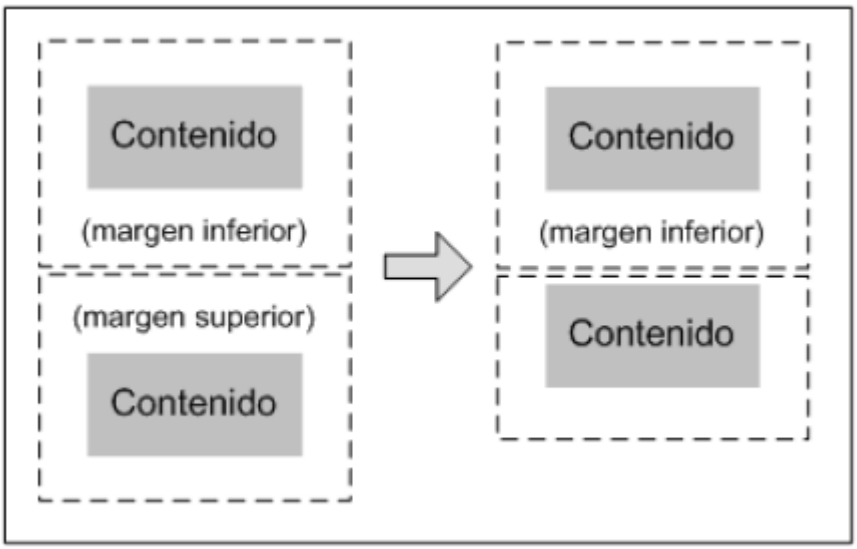
margin-top margin-right margin-bottom margin-left	Margen superior Margen derecho Margen inferior Margen izquierdo
Valores	<medida> | <porcentaje> | auto | inherit
Se aplica a	Todos los elementos, salvo margin-top y margin-bottom que sólo se aplican a los elementos de bloque y a las imágenes
Valor inicial	0
Descripción	Establece cada uno de los márgenes horizontales y verticales de un elemento

margin	Margen
Valores	(<medida> | <porcentaje> | auto) {1, 4} | inherit
Se aplica a	Todos los elementos salvo algunos casos especiales de elementos mostrados como tablas
Valor inicial	-
Descripción	Establece de forma directa todos los márgenes de un elemento

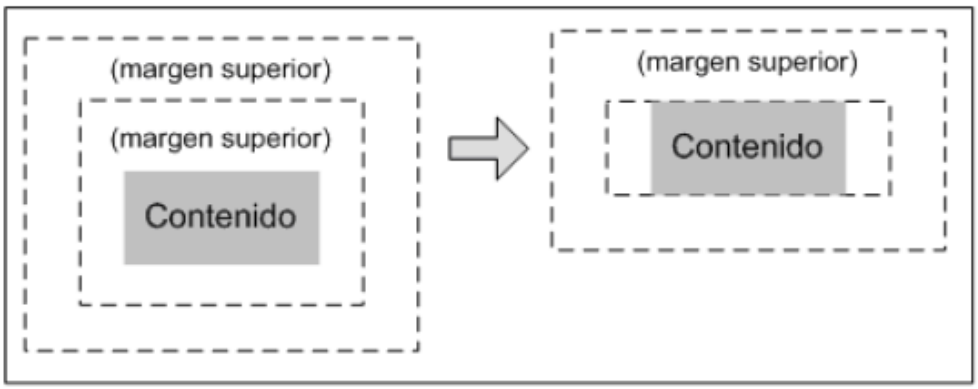
La notación `{1, 4}` de la definición anterior significa que la propiedad `margin` admite entre uno y cuatro valores, con el siguiente significado:

- Si solo se indica un valor, todos los márgenes tienen ese valor.
- Si se indican dos valores, el primero se asigna al margen superior e inferior y el segundo se asigna a los márgenes izquierdo y derecho.
- Si se indican tres valores, el primero se asigna al margen superior, el tercero se asigna al margen inferior y el segundo valor se asigna los márgenes izquierdo y derecho.
- Si se indican los cuatro valores, el orden de asignación es: margen superior, margen derecho, margen inferior y margen izquierdo.

Fusión automática de márgenes verticales :



De la misma forma, si un elemento está contenido dentro de otro elemento, sus márgenes verticales se fusionan y resultan en un nuevo margen de la misma altura que el mayor margen de los que se han fusionado:



Padding :

padding-top **padding-right** **padding-bottom** **padding-left**	Relleno superior Relleno derecho Relleno inferior Relleno izquierdo
Valores	`<medida> | <porcentaje> | inherit`
Se aplica a	Todos los elementos excepto algunos elementos de tablas como grupos de cabeceras y grupos de pies de tabla
Valor inicial	0
Descripción	Establece cada uno de los rellenos horizontales y verticales de un elemento

padding	Relleno
Valores	`( <medida> | <porcentaje> ) {1, 4} | inherit`
Se aplica a	Todos los elementos excepto algunos elementos de tablas como grupos de cabeceras y grupos de pies de tabla
Valor inicial	-
Descripción	Establece de forma directa todos los rellenos de los elementos

Los valores {1, 4} tienen el mismo significado que para los márgenes.

Shorthand . Podemos establecer algunas o todas las propiedades del borde de forma rápida.
Ejemplo :

```
h1 { border-bottom : 1px solid red;   border-top  : 3px solid #FF0001; }
div { border : 3px solid red ; }
```

5. Posicionamiento y visualización.

Capítulo 5 entero de : 02Introduccion_cssLibrosWeb.pdf (páginas 71 a 96). **Resumen** :

Tipos de elementos :
.En bloques (block) ==> <p>
.En líneas (inline) ==> <a>

5 Modos de posicionamiento de una caja :

- Normal o estático. El que utiliza el navegador si no se indica lo contrario.
- Relativo. Se posiciona la caja según su posicionamiento normal y luego se desplaza respecto a esa posición original.
- Absoluto . La posición de la caja se establece de forma absoluta respecto de su elemento contenedor y el resto de elementos de la página ignoran la nueva posición del elemento.
- Fijo. Variante del anterior que convierte la caja en un elemento inamovible aunque el usuario suba o baje la página en el navegador.
-Flotante. Desplaza las cajas hacia la izquierda o derecha de la línea en la que se encuentran.

position	Posicionamiento
Valores	static | relative | absolute | fixed | inherit
Se aplica a	Todos los elementos
Valor inicial	static
Descripción	Selecciona el posicionamiento con el que se mostrará el elemento

El posicionamiento flotante se indica con la palabra **float**.

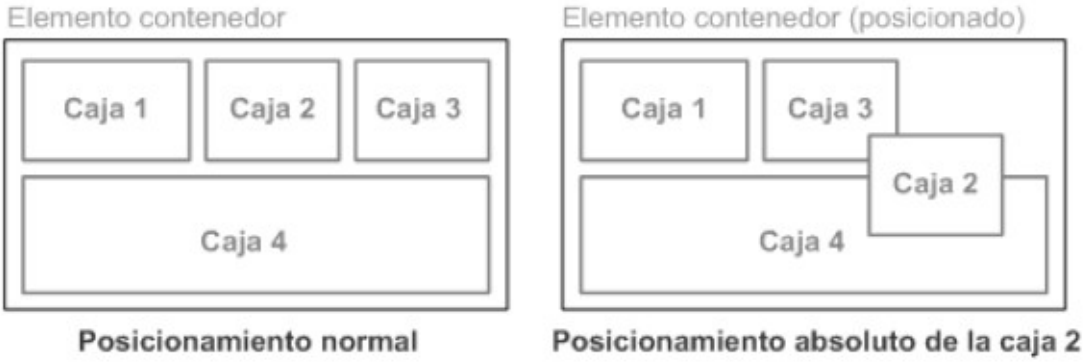
Para el posicionamiento relativo, absoluto y fijo se deben de indicar las propiedades **top**, **right**, **bottom** y **left**, que son las que indicarán la posición de la caja.

Posicionamiento relativo :

El desplazamiento de una caja no afecta al resto de cajas adyacentes, que se muestran en la misma posición que si la caja desplazada no se hubiera movido de su posición original.

Posicionamiento absoluto :

Cuando una caja se posiciona de forma absoluta, el resto de elementos de la página la ignoran y ocupan el lugar original ocupado por la caja posicionada. Al igual que en el posicionamiento relativo, cuando se posiciona de forma absoluta una caja es probable que se produzcan solapamientos con otras cajas.



Posicionamiento fijo :

Cuando una caja se posiciona de forma fija, la forma de obtener el origen de coordenadas para interpretar su desplazamiento es idéntica al posicionamiento absoluto. De hecho, si el usuario no mueve la página HTML en la ventana del navegador, no existe ninguna diferencia entre estos dos modelos de posicionamiento.
La principal característica de una caja posicionada de forma fija es que su posición es inamovible dentro de la ventana del navegador. El posicionamiento fijo hace que las cajas no modifiquen su posición ni aunque el usuario suba o baje la página en la ventana de su navegador.

Posicionamiento flotante :

"`float` fue muy utilizado para maquetar páginas web antes de la llegada de Flexbox y Grid. Hoy en día, ya no se recomienda usarlo para layouts, pero sigue siendo útil para que el texto rodee imágenes u objetos puntuales."

El posicionamiento flotante es el más difícil de comprender pero al mismo tiempo es el más utilizado. La mayoría de estructuras de las páginas web complejas están diseñadas con el posicionamiento flotante.

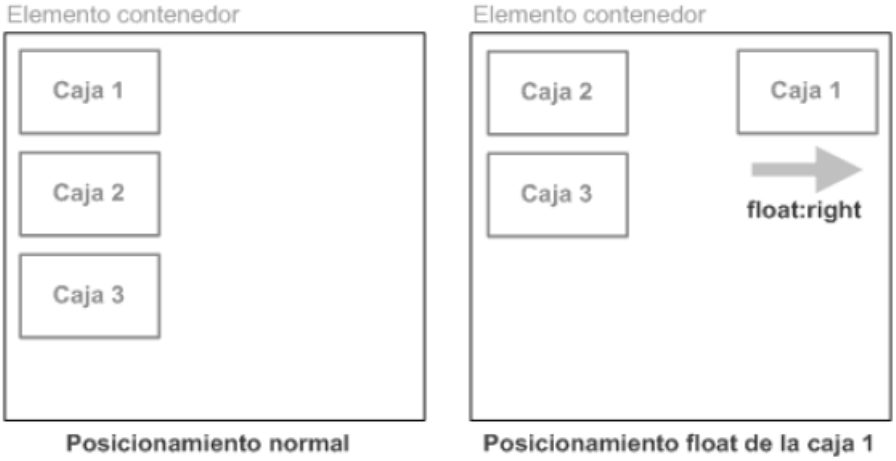


Figura 5.13. Ejemplo de posicionamiento float de una caja

Cuando se posiciona una caja de forma flotante:

- La caja deja de pertenecer al flujo normal de la página, lo que significa que el resto de cajas ocupan el lugar dejado por la caja flotante.
- La caja flotante se posiciona lo más a la izquierda o lo más a la derecha posible de la posición en la que se encontraba originalmente.

Si existen otras cajas flotantes, al posicionar de forma flotante otra caja, se tiene en cuenta el sitio disponible. En el siguiente ejemplo se posicionan de forma flotante hacia la izquierda las tres cajas:

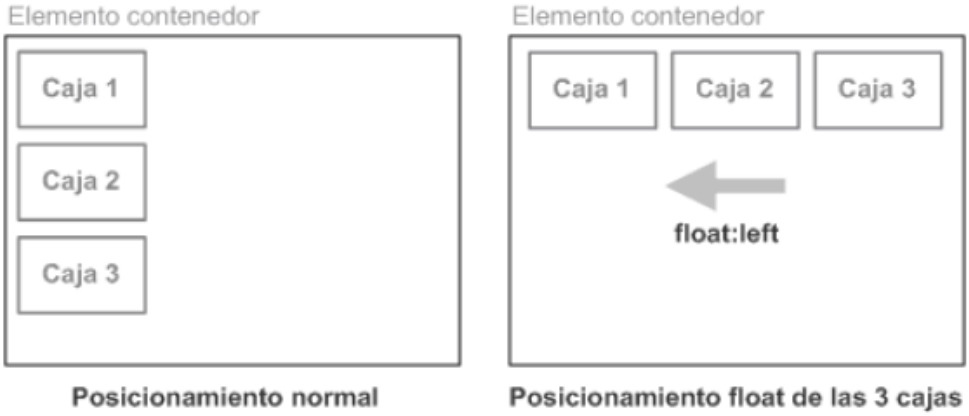


Figura 5.15. Ejemplo de posicionamiento float de varias cajas

En el ejemplo anterior, las cajas no se superponen entre sí porque las cajas flotantes tienen en cuenta las otras cajas flotantes existentes. Como la caja 1 ya estaba posicionada lo más a la izquierda posible, la caja 2 sólo puede colocarse al lado del borde derecho de la caja 1, que es el sitio más a la izquierda posible respecto de la zona en la que se encontraba.

Si no existe sitio en la línea actual, la caja flotante baja a la línea inferior hasta que encuentra el sitio necesario para mostrarse lo más a la izquierda o lo más a la derecha posible en esa nueva línea:

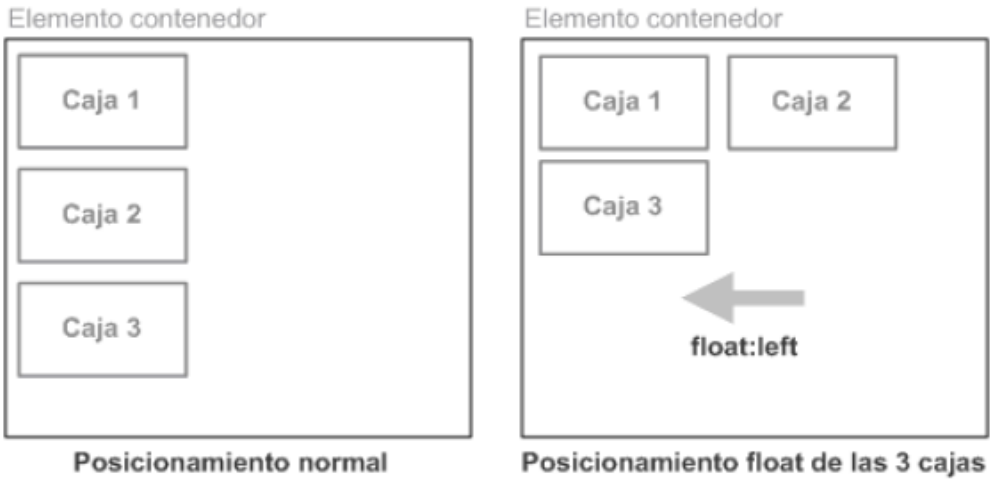


Figura 5.16. Ejemplo de posicionamiento float cuando no existe sitio suficiente

CSS permite controlar la forma en la que los contenidos fluyen alrededor de los contenidos posicionados mediante `float`. De hecho, en muchas ocasiones es admisible que algunos contenidos fluyan alrededor de una imagen, pero el resto de contenidos deben mostrarse en su totalidad sin fluir alrededor de la imagen:

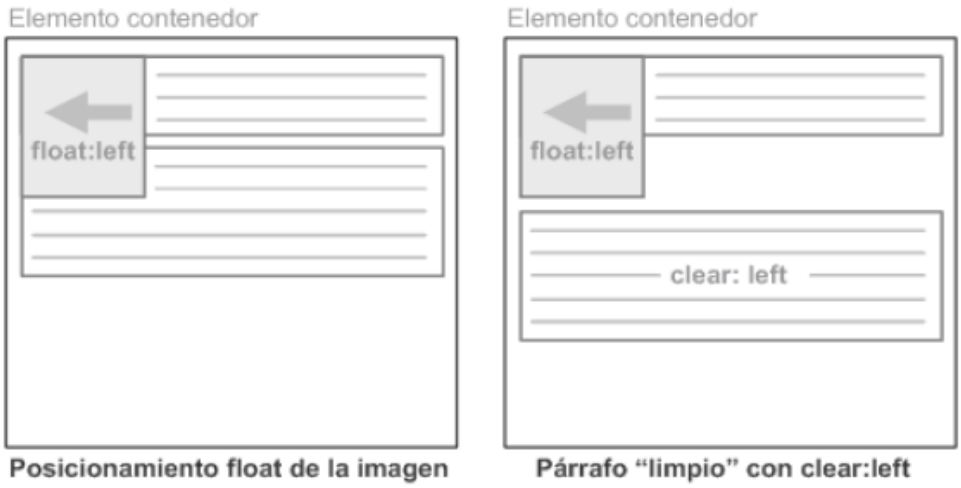


Figura 5.18. Forzando a que un elemento no fluya alrededor de otro elemento posicionado mediante float

clear	Despejar los elementos flotantes adyacentes
Valores	`none` | `left` | `right` | `both` | `inherit`
Se aplica a	Todos los elementos de bloque
Valor inicial	none
Descripción	Indica el lado del elemento que no debe ser adyacente a ninguna caja flotante

La propiedad `clear` indica el lado del elemento HTML que no debe ser adyacente a ninguna caja posicionada de forma flotante. Si se indica el valor `left`, el elemento se desplaza de forma descendente hasta que pueda colocarse en una línea en la que no haya ninguna caja flotante en el lado izquierdo.

La especificación oficial de CSS explica este comportamiento como *"un desplazamiento descendente hasta que el borde superior del elemento esté por debajo del borde inferior de cualquier elemento flotante hacia la izquierda"*.

Si se indica el valor `right`, el comportamiento es análogo, salvo que en este caso se tienen en cuenta los elementos desplazados hacia la derecha.

El valor `both` despeja los lados izquierdo y derecho del elemento, ya que desplaza el elemento de forma descendente hasta que el borde superior se encuentre por debajo del borde inferior de cualquier elemento flotante hacia la izquierda o hacia la derecha.

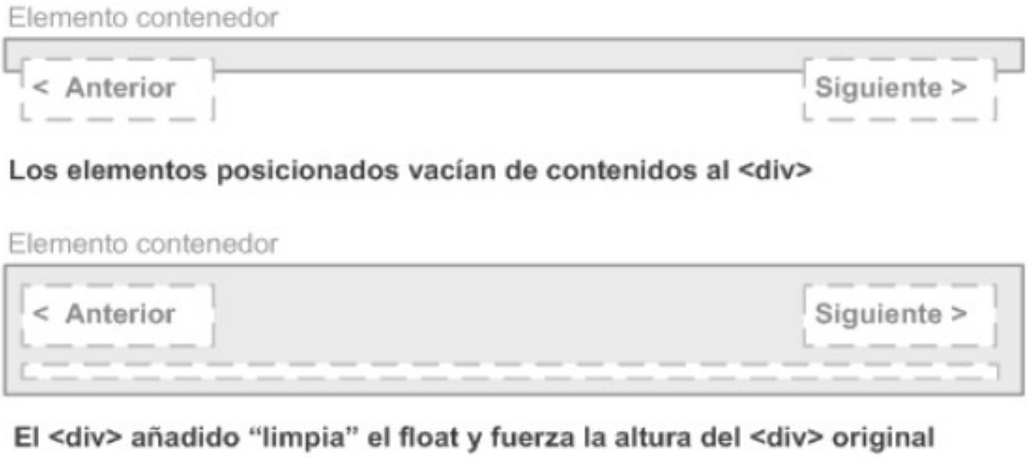


Figura 5.20. Esquema del problema y solución de la visualización incorrecta de dos elementos posicionados mediante float

Visualización :

display, visibility, overflow y z-index .

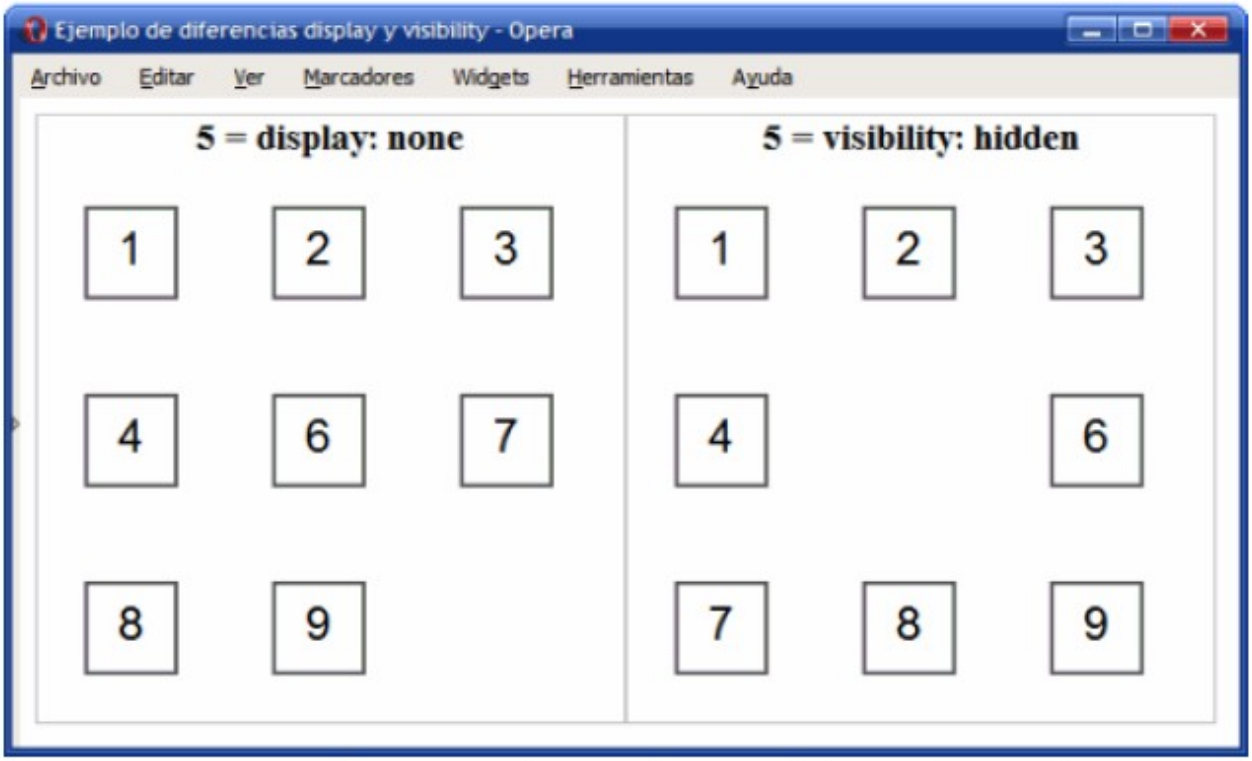


Figura 5.22. Diferencias visuales entre las propiedades display y visibility

A continuación se muestra la definición completa de la propiedad `display`:

display	Visualización de un elemento
Valores	`inline` | `block` | `none` | `list-item` | `run-in` | `inline-block` | `table` | `inline-table` | `table-row-group` | `table-header-group` | `table-footer-group` | `table-row` | `table-column-group` | `table-column` | `table-cell` | `table-caption` | `inherit`
Se aplica a	Todos los elementos
Valor inicial	inline
Descripción	Permite controlar la forma de visualizar un elemento e incluso ocultarlo

Las posibilidades de la propiedad `display` son mucho mayores que ocultar un elemento. En realidad, la propiedad `display` modifica la forma en la que se visualiza un elemento.

Los valores mas utilizados de display son **inline**, **block** e **inline-block**.

El siguiente ejemplo muestra el uso de la propiedad `display` para mostrar un elemento de bloque como si fuera un elemento en línea y para mostrar un elemento en línea como si fuera un elemento de bloque:

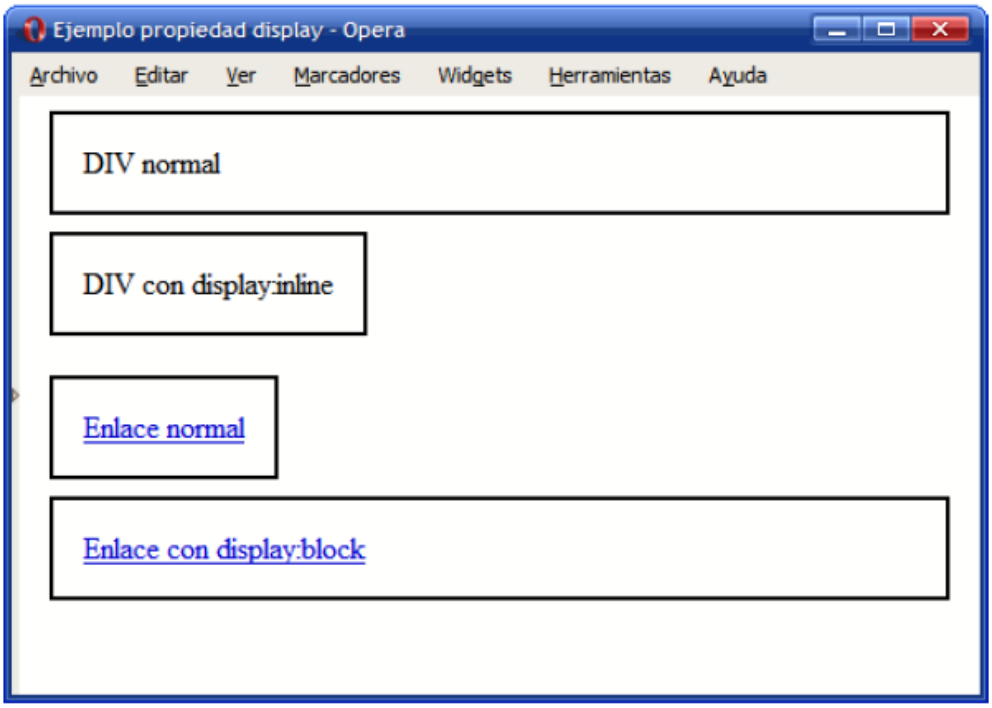


Figura 5.23. Ejemplo de propiedad display

Las reglas CSS del ejemplo anterior son las siguientes:

```
<div>DIV normal</div>
<div style="display:inline">DIV con display:inline</div>

<a href="#">Enlace normal</a>
<a href="#" style="display:block">Enlace con display:block</a>
```

Cuando se establecen las propiedades `display`, `float` y `position` sobre una misma caja, su interpretación es la siguiente:

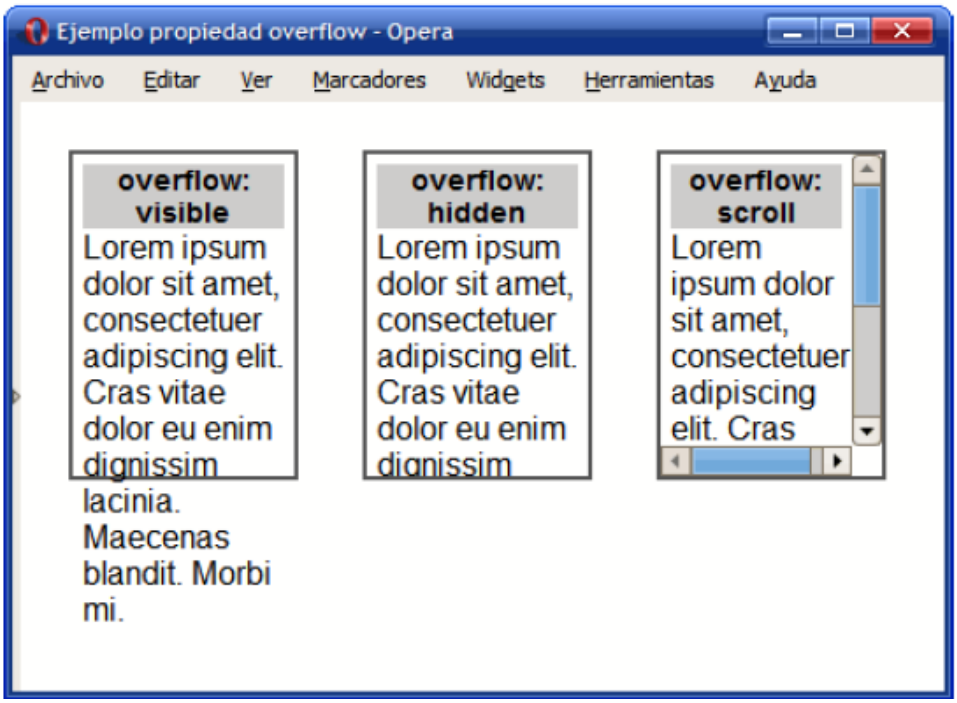
1. Si `display` vale `none`, se ignoran las propiedades `float` y `position` y la caja no se muestra en la página.
2. Si `position` vale `absolute` o `fixed`, la caja se posiciona de forma absoluta, se considera que `float` vale `none` y la propiedad `display` vale `block` tanto para los elementos en línea como para los elementos de bloque. La posición de la caja se determina mediante el valor de las propiedades `top`, `right`, `bottom` y `left`.
3. En cualquier otro caso, si `float` tiene un valor distinto de `none`, la caja se posiciona de forma flotante y la propiedad `display` vale `block` tanto para los elementos en línea como para los elementos de bloque.

overflow	Parte sobrante de un elemento
Valores	visible | hidden | scroll | auto | inherit
Se aplica a	Elementos de bloque y celdas de tablas
Valor inicial	visible
Descripción	Permite controlar los contenidos sobrantes de un elemento

Los valores de la propiedad `overflow` tienen el siguiente significado:

- `visible`: el contenido no se corta y se muestra sobresaliendo la zona reservada para visualizar el elemento. Este es el comportamiento por defecto.
- `hidden`: el contenido sobrante se oculta y sólo se visualiza la parte del contenido que cabe dentro de la zona reservada para el elemento.
- `scroll`: solamente se visualiza el contenido que cabe dentro de la zona reservada para el elemento, pero también se muestran barras de *scroll* que permiten visualizar el resto del contenido.
- `auto`: el comportamiento depende del navegador, aunque normalmente es el mismo que la propiedad `scroll`.

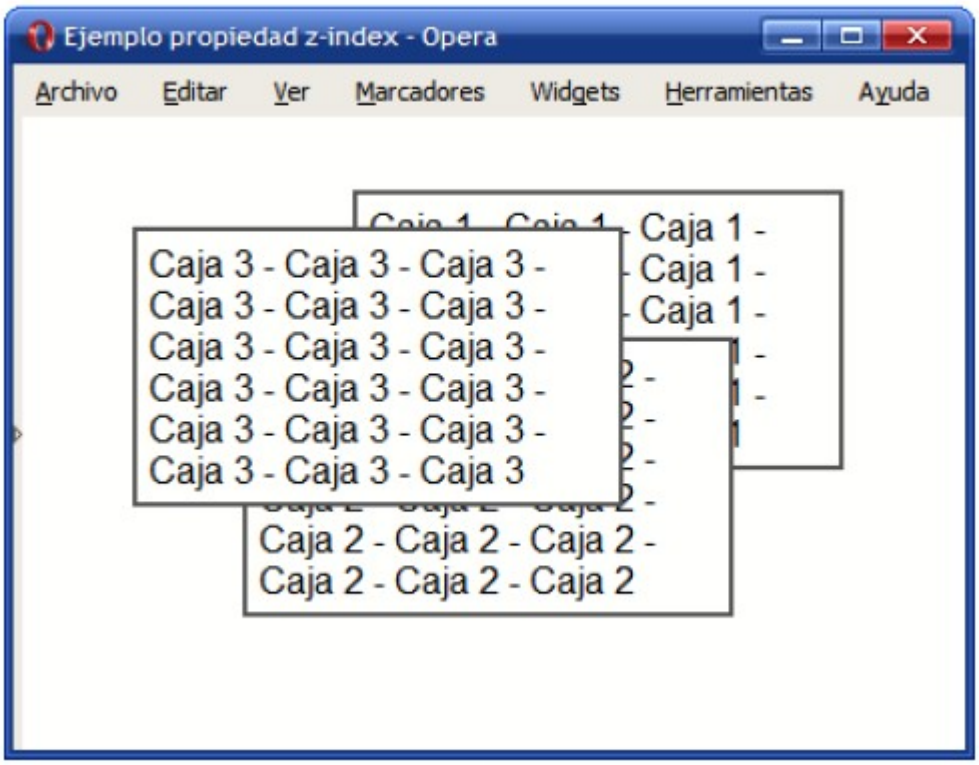
La siguiente imagen muestra un ejemplo de los tres valores típicos de la propiedad `overflow`:



Z-index :

El valor más común de la propiedad `z-index` es un número entero. Aunque la especificación oficial permite los números negativos, en general se considera el número `0` como el nivel más bajo.

Cuanto más alto sea el valor numérico, más cerca del usuario se muestra la caja. Un elemento con `z-index: 10` se muestra por encima de los elementos con `z-index: 8` o `z-index: 9`, pero por debajo de elementos con `z-index: 20` o `z-index: 50`.



Ejemplos :

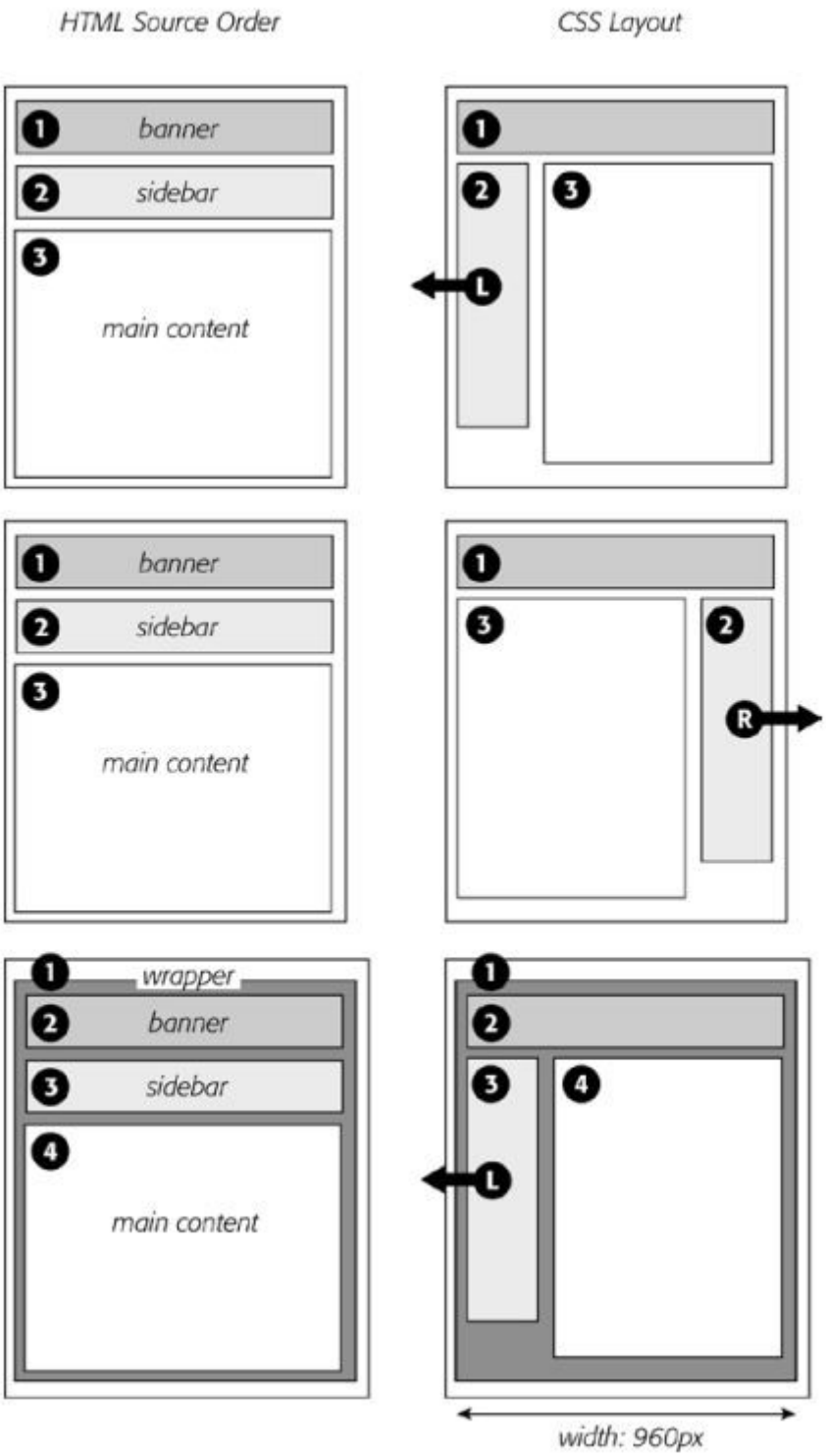
Chia Vet
Because every Chia Pet needs a Chia Vet.
HOME
MEET THE VETS
SERVICES
ABOUT US
DIRECTIONS
CHIA TIPS
B
C
Hotel Chia Vet Opens
How to Prevent Drycoat
Meet Our Newest Doc
A
Lucky Case, Owner of Lulla Chia
Make an appointment: (866) 555-1212
24 Hour Emergency: (866) 555-9111
Chia-Vet.com
700 SW 5th Ave.
Portland, OR 97204
(866) 555-1000

<header class="banner">

padding-top:100px
padding-left:585px

<section class="newsBox">

<footer>


HTML Source Order
CSS Layout
banner
sidebar
main content
L
R
wrapper
width: 960px

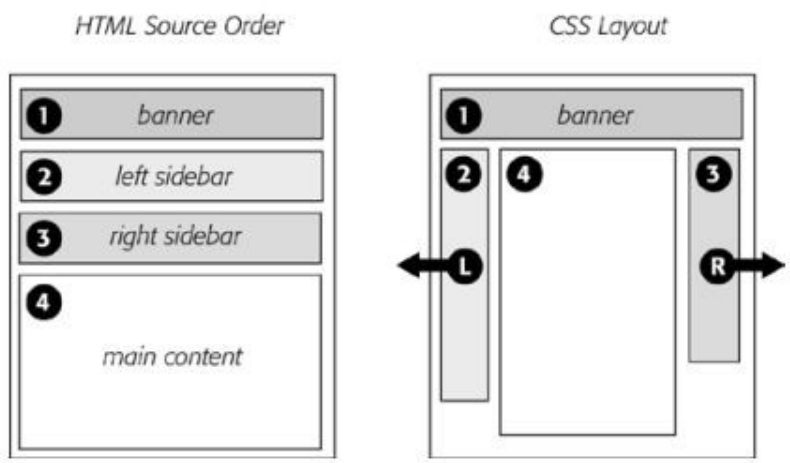


FIGURE 13-3

A three-column design uses the same concepts used to create a two-column design. In this case, you float both the left and right sidebars and add both left and right margins to the center column. The lefthand diagram shows the order of the HTML; the right side shows what the web page looks like.

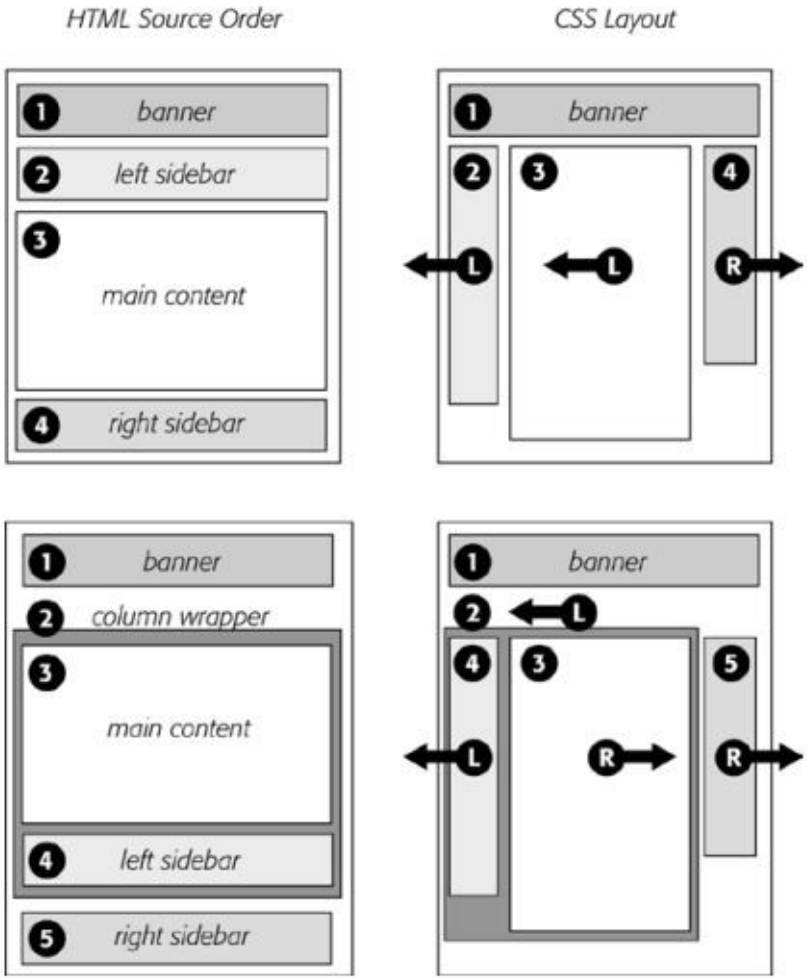


FIGURE 13-4

There's more than one way to float a page. CSS's flexibility provides many ways to create a multicolumn layout. Using different methods of floating, you can easily change the source order of the HTML for the page, as you can see in the diagrams at left. Right-side diagrams represent final web page layout.

6. CSS Grid

Ver cssgrid.pdf

7. Colores, tipografía, texto, fondo, listas

Color

Seleccionamos color de texto :

```
<p style="color:#FF0000";>Rojo texto </p>
<p style="color:Red";>Rojo texto </p>

<p style="color:rgb(255,0,0)";>Rojo texto </p>

<p style="color:rgba(255,0,0,0.5)";>Rojo texto semi transparente </p>

0 = Totalmente transparente, 1 = opaco
```

En ciertas páginas puedes seleccionar un color y te indica los números que debes de poner. Ejemplo: **http://htmlcolorcodes.com/es/**

Los mismos valores anteriores sirven para : background-color, que indica el color de fondo.
Ejemplo :

```
<p style="background-color:rgb(255,0,0)";>Rojo texto </p>
```

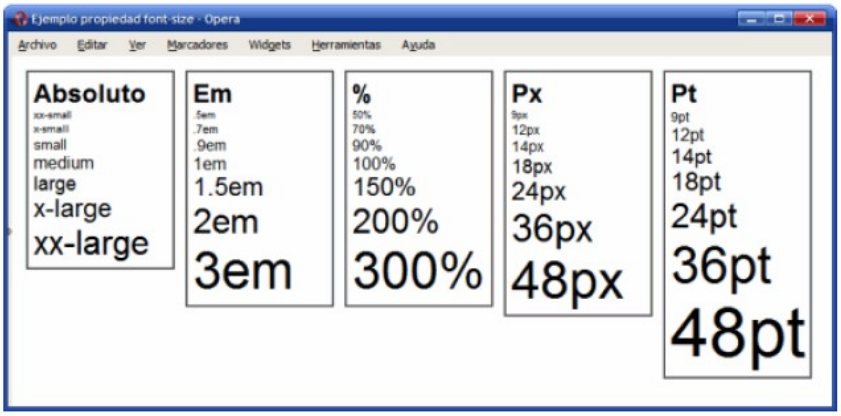
Hay otras formas para indicar color aparte de **rgb** como son **hsl** y **hsla**

Tipografía :

font-family : Indicamos el tipo de letra que deseamos.

```
font-family: "Courier New", Courier, monospace;
font-family: Georgia, "Times New Roman", Times, serif;
font-family: Verdana, Arial, Helvetica, sans-serif;
```

font-size : Indicamos el tamaño.



```
font-size: xx-large;
font-size: 3em;
font-size: 150%;
font-size: 24px;
```

font-style : Estilo que se aplica . Ejemplos :

```
font-style : normal;
font-style : italic;
font-style : oblique;
```

Por otro lado, tenemos el **peso** de la fuente, que no es más que el grosor de la misma. También depende de la fuente elegida, ya que no todas soportan todos los tipos de grosor. De forma similar a como hemos visto hasta ahora, se puede especificar el peso de una fuente mediante tres formas diferentes:

Propiedad	Valor	Significado
font-weight:	**normal** | bold	Medidas absolutas (predefinidas)
font-weight:	bolder | lighter	Medidas relativas (dependen de la actual)
font-weight:	*peso*	Medida específica (número del peso concreto)

- **Valores absolutos**: Normal y bold. **Normal** es el valor por defecto.
- **Valores relativos**: Bolder (más gruesa) o Ligther (más delgada).
- **Valor numérico**: Un número del 100 (menos gruesa) al 900 (mas gruesa). Generalmente, se incrementan en valores de 100 en 100.

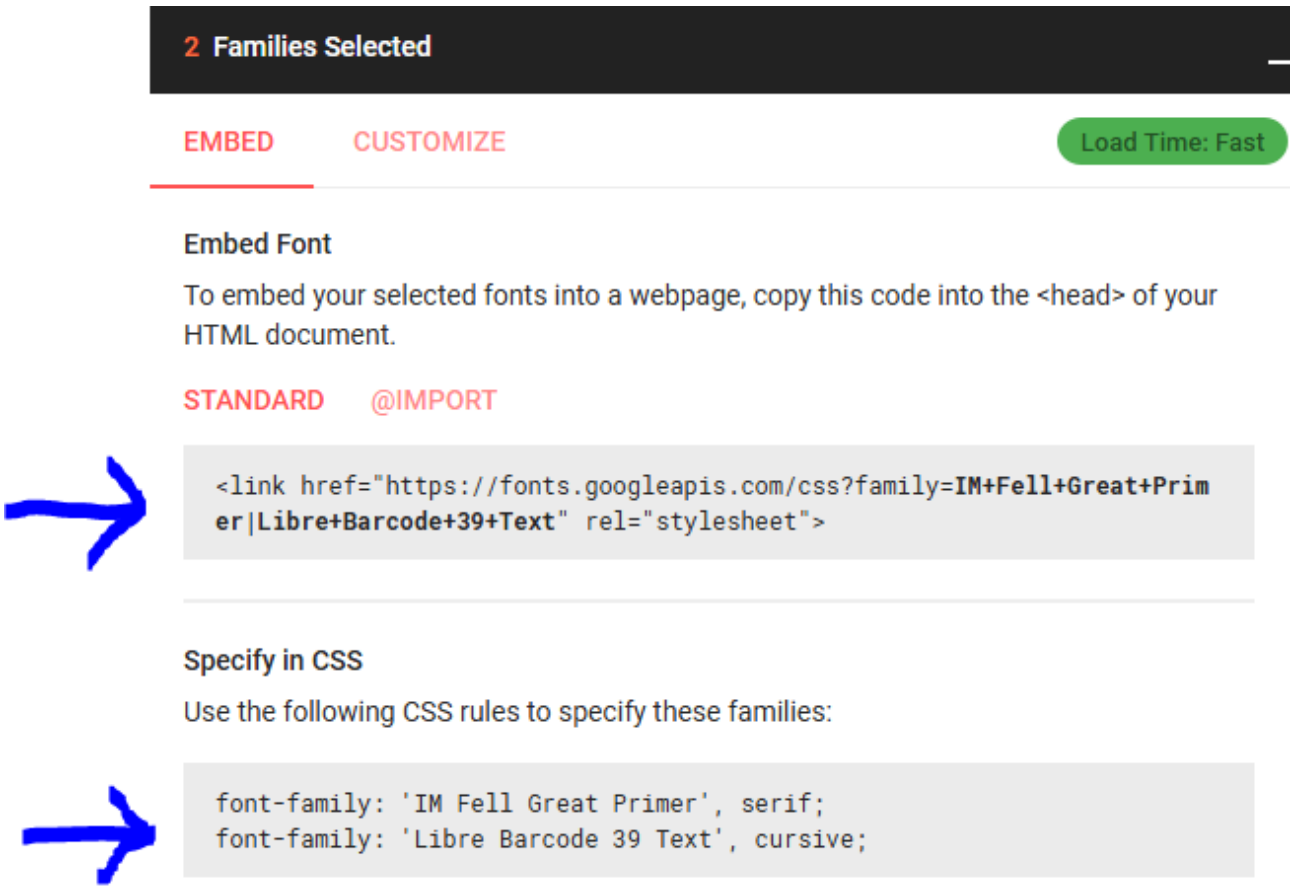
La propiedad font tiene un modo “shorthand”. Ejemplo :

```
font: normal 14px georgia, times, serif; /*Selecciona estilo normal, de 14px y de tipo georgia */
```

(mas información de font en : página 103 de 02Introduccion_cssLibrosWeb.pdf)

Podemos añadir fuentes diferentes a las instaladas en nuestro sistema. Una de las formas de hacerlo es con https://fonts.google.com/

Una vez seleccionada la fuente o fuentes, nos indica lo que tenemos que añadir al archivo html y css para insertar la fuente.



El link se añade dentro del <head> y el CSS dentro del archivo de estilos css.
Otra forma de hacerlo :
http://www.dafont.com/es/ ==> Fuentes disponibles. Las podemos descargar y utilizar luego en nuestras páginas. Ejemplo :

```
@font-face {
      font-family: Los80;
      src: url('fonts/80db.ttf');   /*Descargamos la fuente deseada y la
dejamos en una carpeta */

}
body {
      font-family: Los80;
}
```

http://csstypeset.com/ En esta página podemos probar como quedan nuestras fuentes y el código CSS que se genera.

Texto con sombra. Con la propiedad text-shadow. Podemos visitar la página :

https://css3gen.com/text-shadow/

Donde nos generará el código CSS necesario para generar una sombra en un texto.

Texto :

text-align : left | right | center | justify | inherit ; /*alineación del texto */

line-height : 1.2 | 1.2 em | 120% /* interlineado del texto como múltiplo del tamaño normal de letra */

text-decoration	Decoración del texto
Valores	`none	(underline || overline || line-through || blink)	inherit`
Se aplica a	Todos los elementos
Valor inicial	none
Descripción	Establece la decoración del texto (subrayado, tachado, parpadeante, etc.)

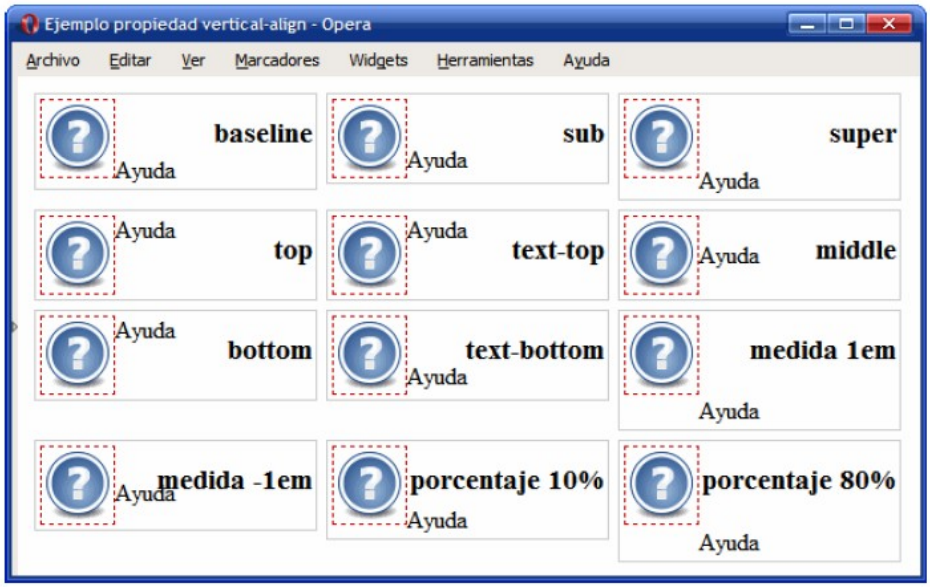
El valor `underline` subraya el texto, por lo que puede confundir a los usuarios haciéndoles creer que se trata de un enlace. El valor `overline` añade una línea en la parte superior del texto, un aspecto que raramente es deseable. El valor `line-through` muestra el texto tachado con una línea continua, por lo que su uso tampoco es muy habitual. Por último, el valor `blink` muestra el texto parpadeante y se recomienda evitar su uso por las molestias que genera a la mayoría de usuarios.

text-transform : capitalize | uppercase | lowercase | none | inherit /* transforma el texto a todo mayúsculas, minúsculas o primera letra en mayúscula (capitalize) */

text-indent : 20px; /* tabulación de la primera línea de párrafo */
letter-spacing : .2em; /* Espaciado entre letras */
word-spacing : .5em; /* Espaciado entre palabras */

`vertical-align`	Alineación vertical
Valores	`baseline` | `sub` | `super` | `top` | `text-top` | `middle` | `bottom` | `text-bottom` | `<porcentaje>` | `<medida>` | `inherit`
Se aplica a	Elementos en línea y celdas de tabla
Valor inicial	baseline
Descripción	Determina la alineación vertical de los contenidos de un elemento

A continuación se muestra una imagen con el aspecto que muestran los navegadores para cada uno de los posibles valores de la propiedad `vertical-align`:



Fondo :

Sobre fondos :
https://mclibre.org/consultar/htmlcss/css/css-fondos.html

Resumen :

```
body {
  background-color: lightblue;  /* Definimos un color de fondo */
}
```

```
body {
   background-image: url("paper.gif"); /* una imagen de fondo */
}
```

Por defecto, background-image repita la imagen horizontal y verticalmente.

```
body {
   background-image: url("gradient_bg.png");
   background-repeat: repeat-x;   /* Se repite horizontalmente, con no-repeat no se repetiría */
}
```

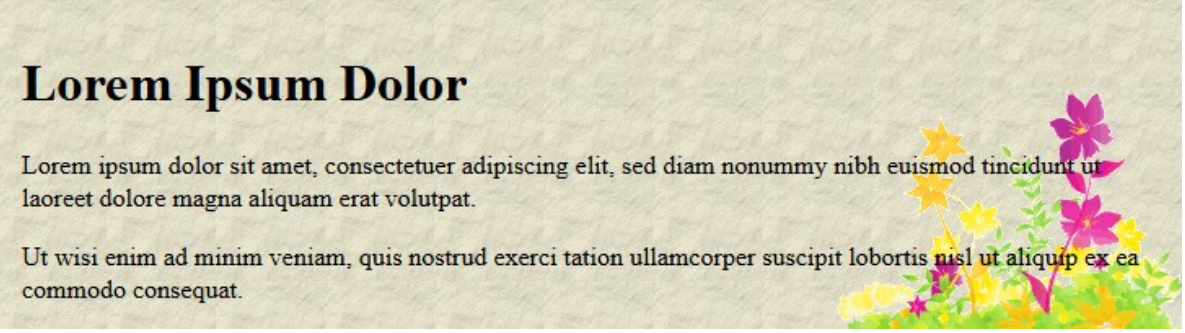
```
body {
   background-image: url("img_tree.png");
   background-repeat: no-repeat;
   background-position: right top; /* Posicionamos la imagen arriba a la derecha */
}
```

```
body {
   background-image: url("img_tree.png");
   background-repeat: no-repeat;
   background-position: right top;
   margin-right: 200px;
   background-attachment: fixed; /*Fijamos la imagen, no se mueve aunque lo haga la página */
}
```

CSS3 permite varios fondos :

```
#example1 {
 background-image: url(flores.gif), url(papel.gif);
 background-position: right bottom, left top;
 background-repeat: no-repeat, repeat;
}
```

Quedaría algo como :



También permite ajustar el tamaño de la imagen :

```
#div1 {
 background: url(img_flower.jpg);
 background-size: 100px 80px;   // 100% 100%; (Otros valores posibles)
 background-repeat: no-repeat;
}
```

background-size ==> Puede contener las palabras contain y cover.
contain ==> Se ajusta el tamaño al contenedor, manteniendo las proporciones (puede haber partes del contenedor no cubiertas por la imagen)
cover ==> Cubre todo el contenido manteniendo la proporción de la imagen. Puede ser que algunas partes de la imagen no se vean en el contenido.

En el siguiente ejemplo, cubrimos toda la página con una imagen de fondo.

```
html {
 background: url(img_flower.jpg) no-repeat center fixed;
 background-size: cover;
}
```

background-origin: ==> Puede contener 3 valores :

- border-box – La imagen empieza en la esquina superior izquierda del borde
- padding-box – La imagen empieza en la esquina superior izquierda del padding
- content-box - La imagen empieza en la esquina superior izquierda del contenido

background-clip: ==> Especifica el área de pintado de un background. Toma los mismos valores que la propiedad anterior.

Listas :

Propiedad	**list-style-type**
Valores	disc | circle | square | decimal | decimal-leading-zero | lower-roman | upper-roman | lower-greek | lower-latin | upper-latin | armenian | georgian | lower-alpha | upper-alpha | none | inherit
Se aplica a	Elementos de una lista
Valor inicial	disc
Descripción	Permite establecer el tipo de viñeta mostrada para una lista

La siguiente imagen muestra algunos de los valores definidos por la propiedad `list-style-type`:

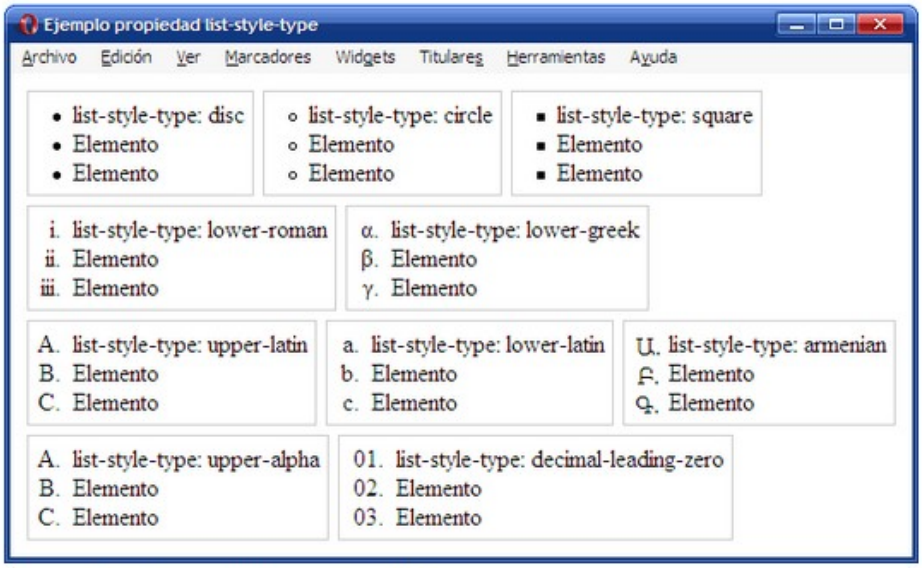


Figura 9.1 Ejemplo de propiedad list-style-type

El código CSS de algunas de las listas del ejemplo anterior se muestra a continuación:

```
<ul style="list-style-type: square">
  <li>list-style-type: square</li>
  <li>Elemento</li>
  <li>Elemento</li>
</ul>
```

Propiedad	**list-style-position**
Valores	inside | outside | inherit
Se aplica a	Elementos de una lista
Valor inicial	outside
Descripción	Permite establecer la posición de la viñeta de cada elemento de una lista

La diferencia entre los valores `outside` y `inside` se hace evidente cuando los elementos contienen mucho texto, como en la siguiente imagen:

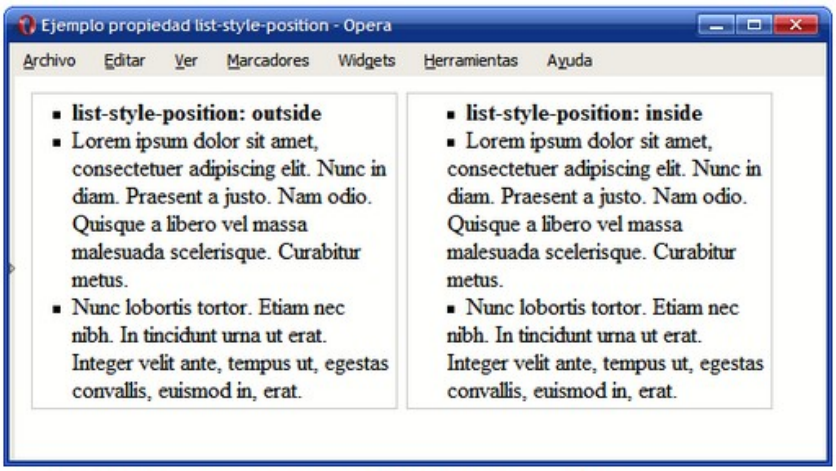


Figura 9.2 Ejemplo de propiedad list-style-position

Propiedad	**list-style-image**
Valores	url | none | inherit
Se aplica a	Elementos de una lista
Valor inicial	none
Descripción	Permite reemplazar las viñetas automáticas por una imagen personalizada

Las imágenes personalizadas se indican mediante la URL de la imagen. Si no se encuentra la imagen o no se puede cargar, se muestra la viñeta automática correspondiente (salvo que explícitamente se haya eliminado mediante la propiedad `list-style-type`).

La siguiente imagen muestra el uso de la propiedad `list-style-image` mediante tres ejemplos sencillos de listas con viñetas personalizadas:

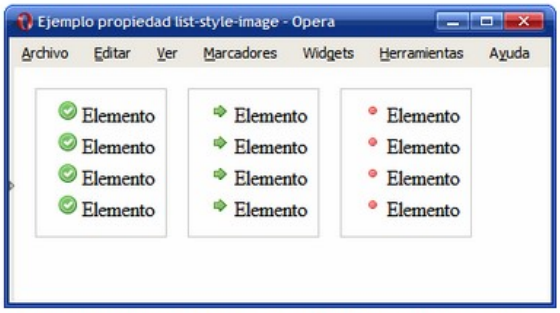


Figura 9.3 Ejemplo de propiedad list-style-image

Las reglas CSS correspondientes al ejemplo anterior se muestran a continuación:

```
ul.ok { list-style-image: url("imagenes/ok.png"); }
ul.flecha { list-style-image: url("imagenes/flecha.png"); }
ul.circulo { list-style-image: url("imagenes/circulo_rojo.png"); }
```

8. Avanzado : botones, degradados, menús, tablas, transformaciones, etc

Prefijos -moz- , -webkit- , -o-, -ms-

https://css3gen.com/ Para generar : texto con sombra, botones, degradados, cajas con sombra, etc.

Para mas información sobre gradientes : https://www.w3schools.com/css/css3_gradients.asp

El **menú** en html podría ser algo como :

```
<div class="scrollmenu">
  <a href="#home">Home</a>
  <a href="#news">Noticias</a>
  <a href="#contact">Contacto</a>
  <a href="#about">Acerca</a>
  <a href="#support">Soporte</a>
</div>
```

Luego le podemos asociar un código CSS como el siguiente :

```
<style>
div.scrollmenu {
    background-color: #333; // color de fondo a todo el menú
}

div.scrollmenu a {
    display: inline-block;
    color: white;
    text-align: center;
    padding: 14px;
    text-decoration: none;
}

div.scrollmenu a:hover {
    background-color: #777;
}
```

De esta forma generamos un menú horizontal. Aplicándole el siguiente CSS lo que conseguimos es que el menú sea vertical.

```
<style>
.scrollmenu {
```

```
    width: 200px;
}

.scrollmenu a {
    background-color: #eee;
    color: black;
    display: block;
    padding: 12px;
    text-decoration: none;
}

.scrollmenu a:hover {
    background-color: #ccc;
}

.scrollmenu a.active {
    background-color: #4CAF50;
    color: white;
}
```

Para generar menús :

http://www.cssmenubuilder.com/home

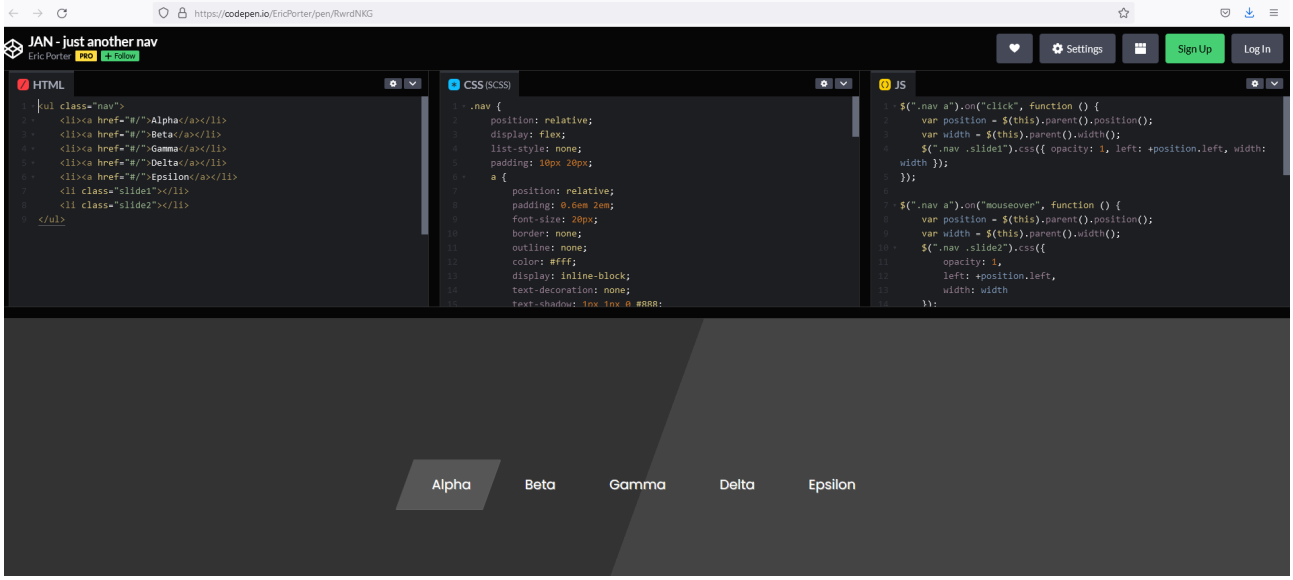
http://www.cssportal.com/css3-menu-generator/

Tecleando “plugin menu jquery” en google podemos obtener lista de páginas donde se habla de una serie de menús realizados con jquery (librería para javascript). Una de ellas es :

https://freefrontend.com/jquery-menus/

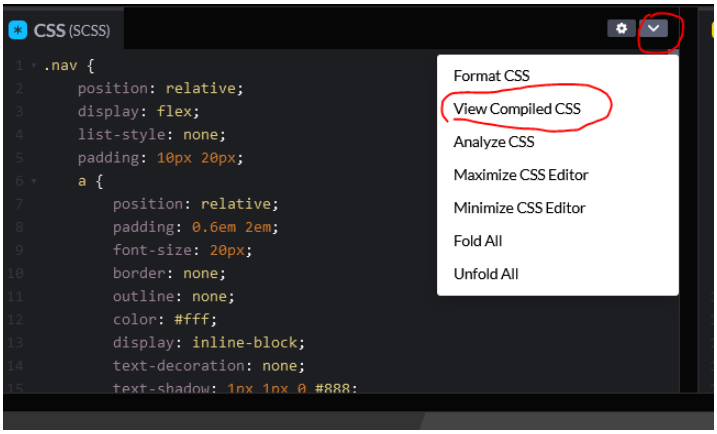
Veamos uno de ellos : Just Another Nav

Nos lleva a una página (codepen.io) donde se muestra :



- El código HTML

- El código CSS o SCSS. Necesitamos el CSS con lo que deberemos :



- El código JS necesario para que realice la funcionalidad.

- Como funciona el menú (en la parte inferior)

Para que funcione hay que descargar (o hacer enlace) a la librería de jQuery de esta forma :

(https://code.jquery.com/)

```
<script                    src="https://code.jquery.com/jquery-3.6.0.js"
```

```
integrity="sha256-H+K7U5CnXl1h5ywQfKtSj8PCmoN9aaq30gDh27Xc0jk="
crossorigin="anonymous"></script>
```

Observando los ejemplos podemos modificar elementos aún sin conocer jQuery en profundidad. Lo mínimo que deberemos saber de jQuery es :

$ ==> Indica que es jQuery

$(".nav a") ==> Lo que va entre paréntesis es el código CSS que hace referencia a uno o varios elementos HTML.

Y si no funciona… :

- Asegurarse de que el enlace a jQuery se realizó correctamente.

- El CSS es el correcto (y no versión SCSS)

- El código JS puede hacer referencia a elementos que todavía no están cargados en la página, con lo que no funciona correctamente. Para arreglar esto :

1) Insertar el código JS al final de la página

2) Mucho mejor : Incluir todo el código JS dentro de :

```
$( document ).ready(function() {
        // todo el código JS de la página aquí dentro.
});
```

Tablas :

A una tabla del tipo :

```
<table>
  <tr>
    <th>Nombre</th>
    <th>Apellido</th>
    <th>Importe</th>
  </tr>
  <tr>
    <td>Peter</td>
    <td>Griffin</td>
    <td>$100</td>
  </tr>
  <tr>
    <td>Lois</td>
```

```
    <td>Griffin</td>
    <td>$150</td>
  </tr>
  <tr>
    <td>Joe</td>
    <td>Swanson</td>
    <td>$300</td>
  </tr>
  <tr>
    <td>Cleveland</td>
    <td>Brown</td>
    <td>$250</td>
</tr>
</table>
```

Le podemos dar formato CSS de la siguiente forma :

```
<style>
table, td, th {
    border: 1px solid black;
}

table {
    border-collapse: collapse;
    width: 100%;
}

th {
    height: 50px;
    background-color: green;
}
td {
       background-color: #CCCCCC;
}
</style>
```

El resultado sería :

Nombre	Apellidos	Importe
Peter	Griffin	$100
Lois	Griffin	$150
Joe	Swanson	$300
Cleveland	Brown	$250

Para ampliar la teoría sobre tablas CSS :

http://aprende-web.net/css/css6_2.php y

http://aprende-web.net/css/css6_3.php

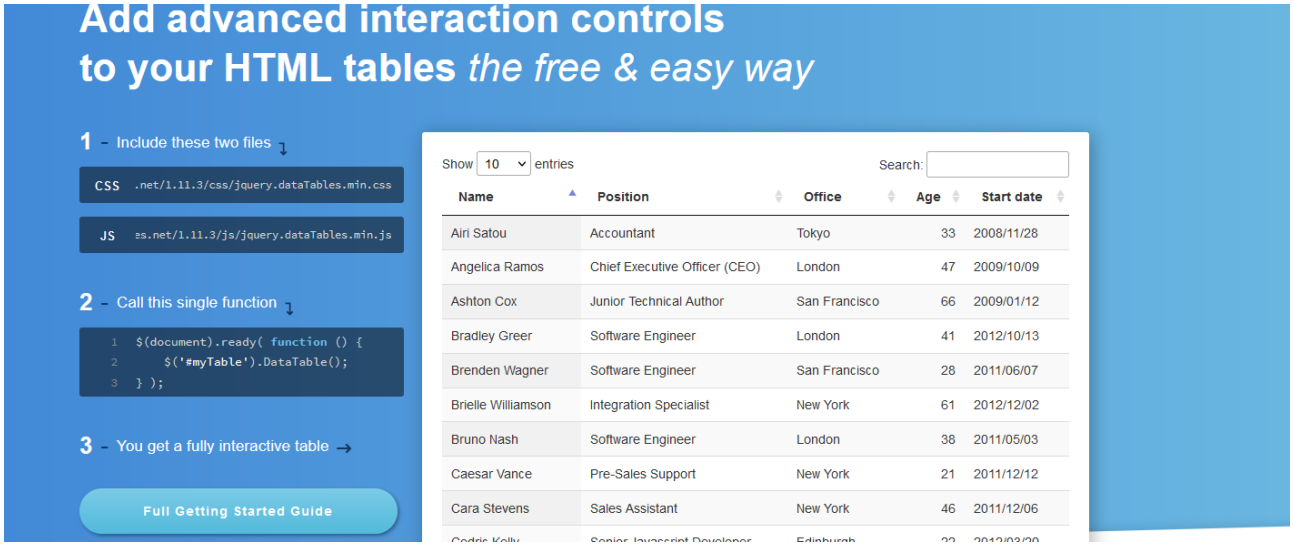
Para generar tablas con CSS :

http://tablestyler.com/

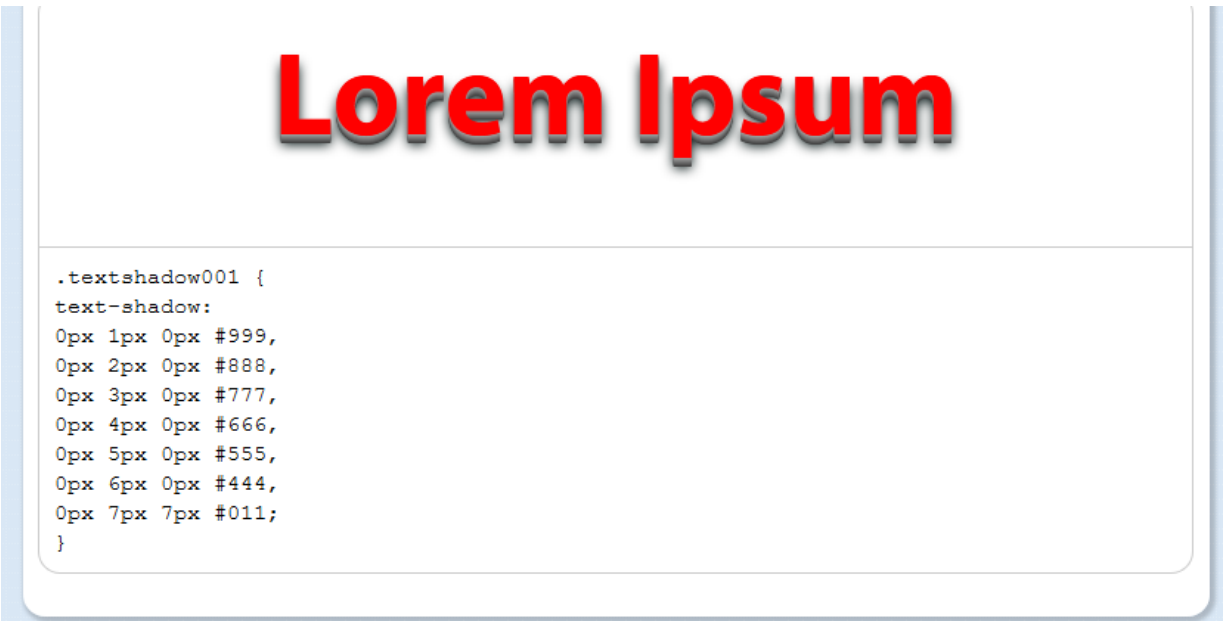
Buscando plugins para tablas en google encontramos :

https://datatables.net/

El funcionamiento es prácticamente idéntico a los menús. Siguiendo los ejemplos deberemos conseguir que una tabla tenga la funcionalidad que deseemos.



Observa el siguiente ejemplo de : http://www.css3-generator.de/text-shadow.html



Imágenes :

https://www.w3schools.com/css/css3_images.asp

Podemos redondear las imágenes con

```
img {
       border-radius: 8px; // Probar con 50% sobre una imagen
}
<style>
img {
    border: 1px solid #ddd;
    border-radius: 4px;
    padding: 5px;
    width: 150px;
}

img:hover {
    box-shadow: 0 0 2px 1px rgba(0, 140, 186, 1);
}
```

Busca box-shadow en w3schools.com para ver como funciona.

De la siguiente forma podrás tener imágenes responsive :

```
img {
    max-width: 100%;
    height: auto;
}
```

Podemos centrar una imagen :

```
img {
 display: block;
 margin: auto;
 width: 50%;
}
```

Podemos establecer la opacidad de la imagen. 1 = totalmente opaca.

```
img {
 opacity: 0.5;
 filter: alpha(opacity=50); /* For IE8 and earlier */
}
```

Aplicar filtros sobre la imagen :

```
img {
 -webkit-filter: grayscale(100%); /* Safari 6.0 - 9.0 */
 filter: grayscale(100%);  // Escala de grises
}
```

Otros filtros :

```
.blur {-webkit-filter: blur(4px);
       filter: blur(4px);}
.brightness {-webkit-filter: brightness(250%);
       filter: brightness(250%);}
.contrast {-webkit-filter: contrast(180%);
       filter: contrast(180%);}
.huerotate {-webkit-filter: hue-rotate(180deg);
       filter: hue-rotate(180deg);}
.invert {-webkit-filter: invert(100%);
       filter: invert(100%);}
.opacity {-webkit-filter: opacity(50%);
       filter: opacity(50%);}
.saturate {-webkit-filter: saturate(7);
       filter: saturate(7);}
.sepia {-webkit-filter: sepia(100%);
       filter: sepia(100%);}
.shadow {-webkit-filter: drop-shadow(8px 8px 10px green);
       filter: drop-shadow(8px 8px 10px green);}
```

El funcionamiento de estos filtros los podrás ver en :
`https://www.w3schools.com/css/tryit.asp?filename=trycss_ex_images_filters`

Sobre imágenes en CSS3 : https://www.w3schools.com/css/css3_images.asp

Transformaciones 2D.

Ver : **EjemploTransformaciones.html**

```
div {
-ms-transform: translate(50px, 100px); /* IE 9 */
-webkit-transform: translate(50px, 100px); /* Safari */
transform: translate(50px, 100px);
}
```

transform: translate(50px, 100px); ==> Mueve el elemento a la dcha y hacia abajo

```
div {
 -ms-transform: rotate(20deg); /* IE 9 */
 -webkit-transform: rotate(20deg); /* Safari */
 transform: rotate(20deg);
}
```

transform: rotate(20deg); ==> rota el elemento los grados indicados (si > 0 en sentido agujas del reloj)

transform: scale(2, 3); ==> Escala el elemento 2 veces su widht y 3 su height

Transformaciones 3D.

Ver : **EjemploTransformaciones3D.html**

transform: rotateX(150deg); ==> Rota sobre el eje X los grados indicados
transform: rotateY(130deg); ==> Rota sobre el eje Y
transform: rotateZ(90deg); ==> Rota sobre el eje Z

Transiciones

Ver : **EjemploTransiciones.html**

Permiten modificar el valor de una propiedad de forma gradual durante un tiempo.
Ejemplo :

```
div {
   width: 100px;
```

```
   height: 100px;
   background: red;
   -webkit-transition: width 2s; /* For Safari 3.1 to 6.0 */
   transition: width 2s; /* propiedad afectada y tiempo duración */
}

div:hover {
   width: 300px;
}
```

El div anterior tiene asociado un witdth. Con transition le indiacamos el atributo a modificar y el tiempo que dura esa transformación. Luego, con el :hover indicamos el nuevo valor del width. Con ello logramos que al pasar el ratón por encima, el ancho del div aumentará hasta los 300px y durante 2 segundos se verá como, poco a poco, el tamaño va aumentando.

transition: width 2s, height 4s; ==> Podemos indicar varios atributos distintos

transition-timing-function ==> Define la cuva del movimiento

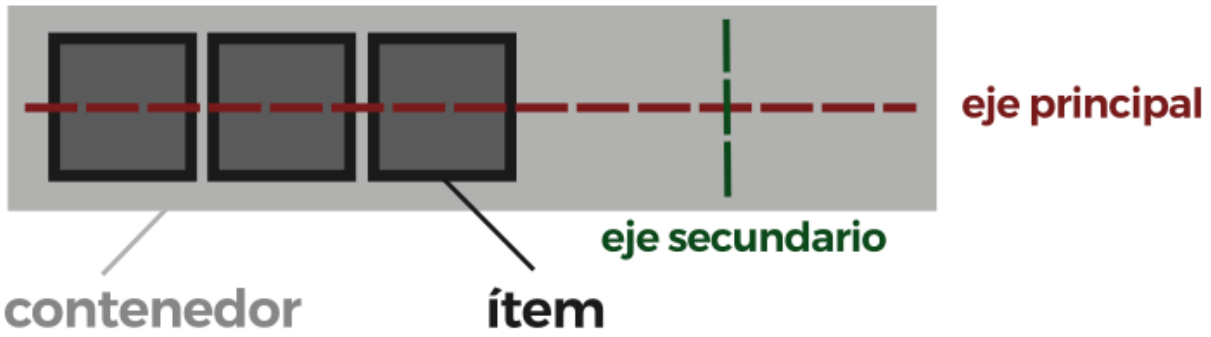
#div1 {transition-timing-function: linear;} ==> Misma velocidad siempre
#div2 {transition-timing-function: ease;} ==> despacio al inicio, rapido, despacio al final
#div3 {transition-timing-function: ease-in;} ==> despacio al inicio
#div4 {transition-timing-function: ease-out;} ==> despacio al final
#div5 {transition-timing-function: ease-in-out;} despacio al inicio y final

transition: width 2s, height 2s, transform 2s; ==> Podemos indicar un tiempo de transición a una transformación también : transform: rotate(180deg);

transition-delay: 1s; ==> Un retardo antes de iniciar la transición

9. Flexbox

Es un sistema de posicionamiento mas potente, límpia y personalizable.



Contenedor: Existe un elemento padre que es el contenedor que tendrá en su interior cada uno de los ítems flexibles y adaptables.

Ítem: Cada uno de los hijos flexibles que tendrá el contenedor en su interior.

Eje principal: Los contenedores flexibles tendrán una orientación principal específica. Por defecto, es en horizontal (fila).

Eje secundario: De la misma forma, los contenedores flexibles tendrán una orientación secundaria, perpendicular a la principal. Si la principal es en horizontal, la secundaria será en vertical, y viceversa.

Supongamos que tenemos esto :

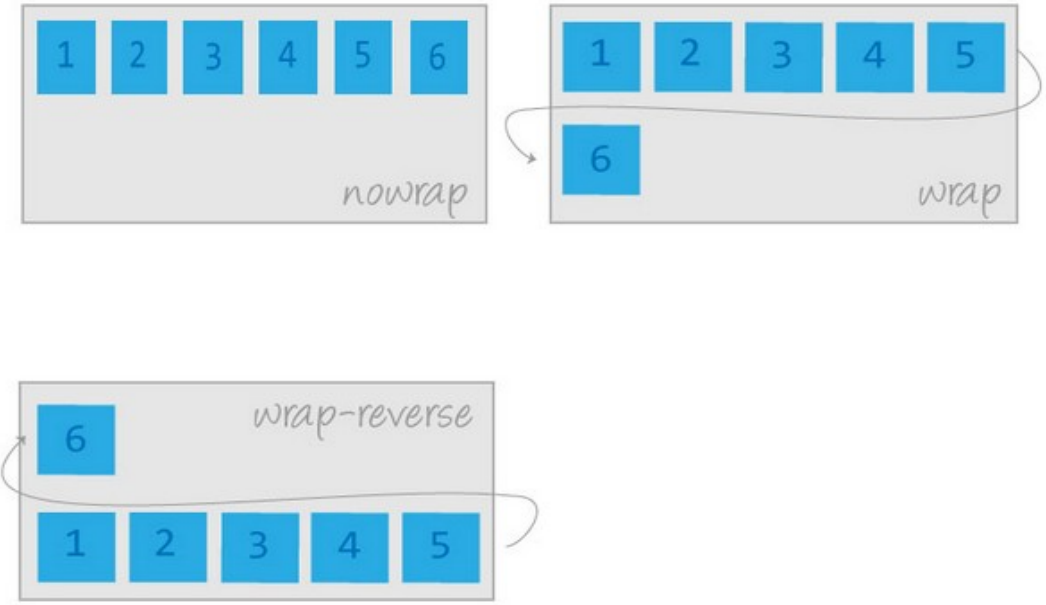
```
<div id="contenedor"> <!-- contenedor flex -->
  <div class="item item-1">1</div> <!-- cada uno de los ítems flexibles -->
  <div class="item item-2">2</div>
  <div class="item item-3">3</div>
</div>
```

Para activar el modo flexbox hay que hacer display : flex (contenedor de items en bloque) o inline-flex (contenedor de items en línea – como inline-block) en el elemento contenedor.

Con hacer eso, los items se dispondrán todos en una misma fila.

Con flex-direction se puede indicar la dirección del eje principal, es decir, la fila o columna. Valores posibles : row, column, row-reverse, column-reverse.

Con flex-wrap indicamos si queremos o no desbordamiento en caso necesario. Valores posibles : wrap, no-wrap, wrap-reverse. De esta forma podemos determinar si es o no multilínea.



Podemos ver un ejemplo en : EjemploHTMLFlexbox0.html

Existe una propiedad atajo (short-hand) :

```
flex-flow: <flex-direction> <flex-wrap>;
```

```
flex-flow: row wrap;
```

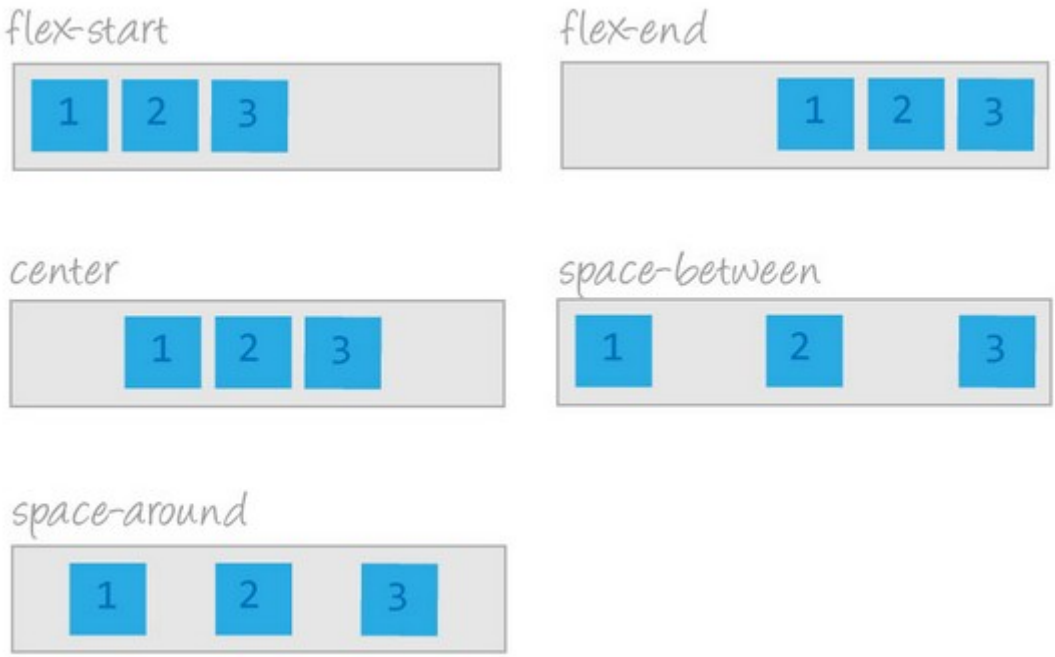
Vamos a ver ahora como disponer los ítems dentro del contenedor. Cuatro propiedades para ello :

Propiedad	Valor	Actúa sobre
justify-content:	**flex-start** | flex-end | center | space-between | space-around	Eje principal
align-content:	flex-start | flex-end | center | space-between | space-around | **stretch**	Eje principal
align-items:	flex-start | flex-end | center | **stretch** | baseline	Eje secundario
align-self:	**auto** | flex-start | flex-end | center | stretch | baseline	Eje secundario

Observa que hay dos propiedades que actúan sobre el eje principal y otras dos sobre el eje secundario. Recuerda que el eje principal es horizontal si la propiedad flex-direction es row .

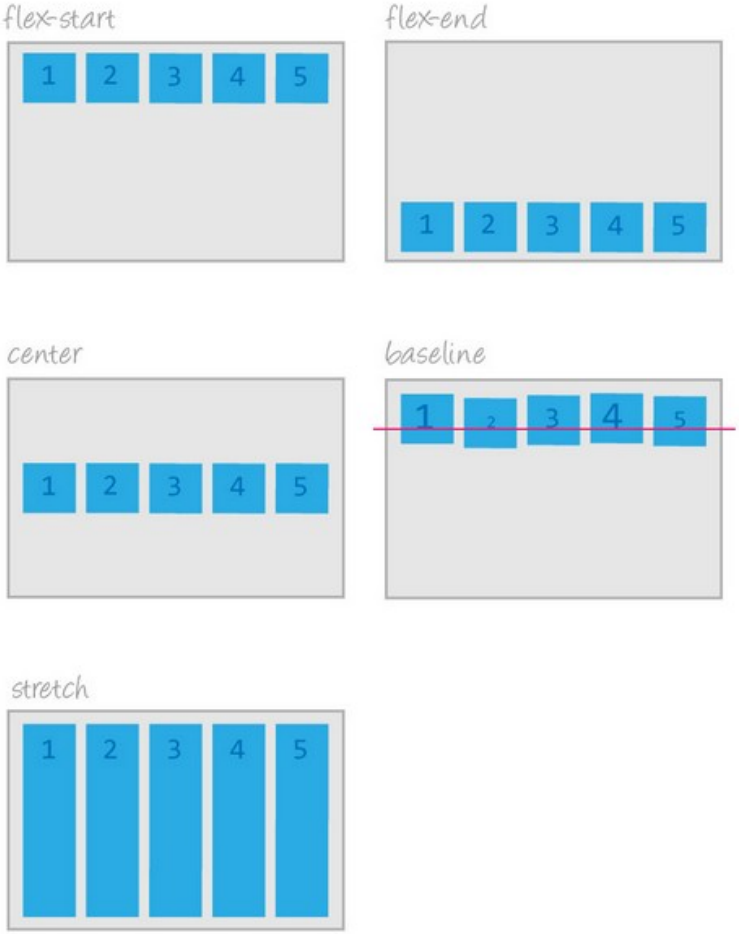
justify-content: Se utiliza para alinear los ítems del **eje principal** (por defecto, el horizontal)

```
justify-content: flex-start | flex-end | center | space-between | space-around
```



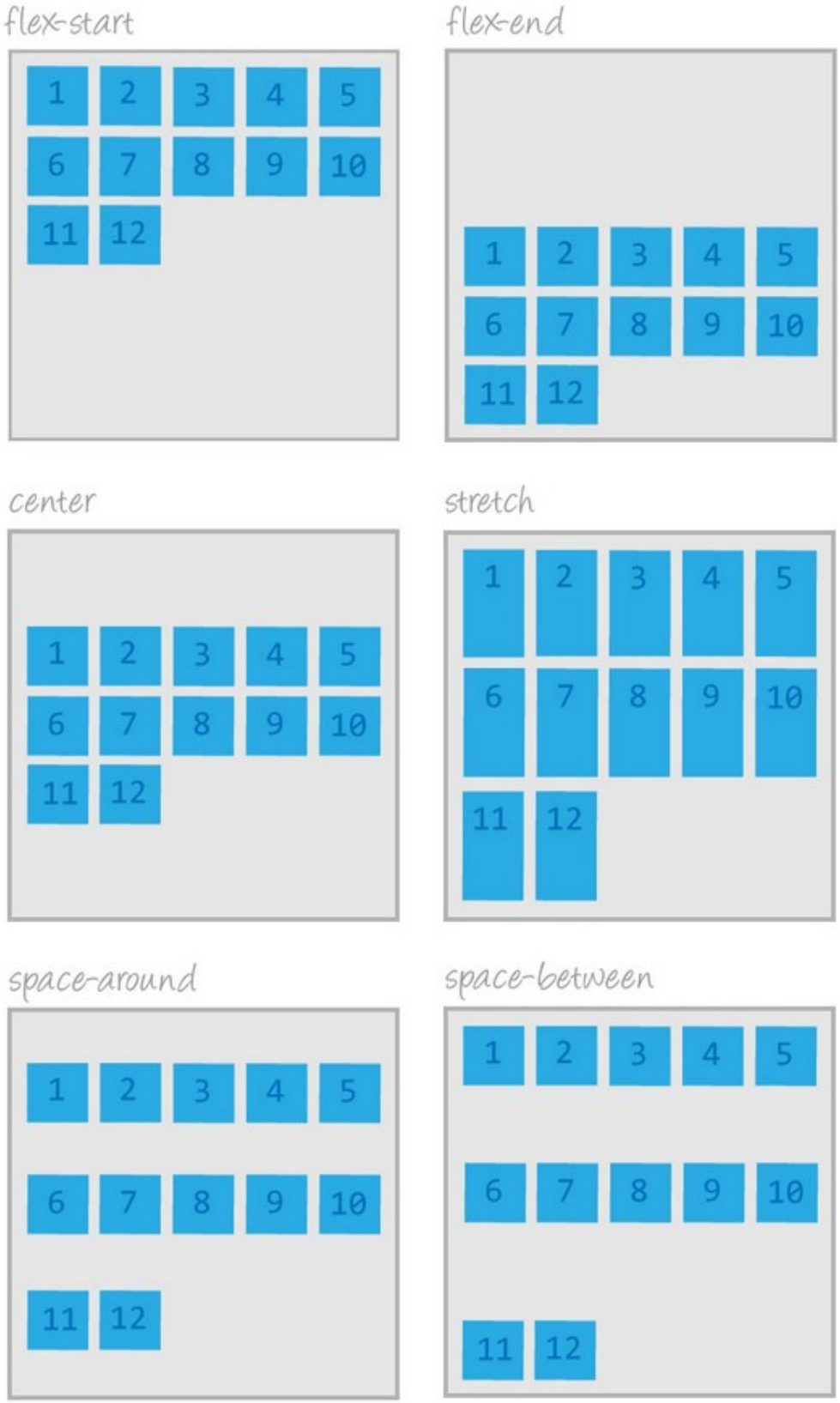
align-items: Usada para alinear los ítems del **eje secundario** (por defecto, el vertical).

```
align-items: flex-start | flex-end | center | baseline | stretch
```



align-content para alinear múltiples líneas. No tiene efecto cuando el contenedor contiene una sola línea.

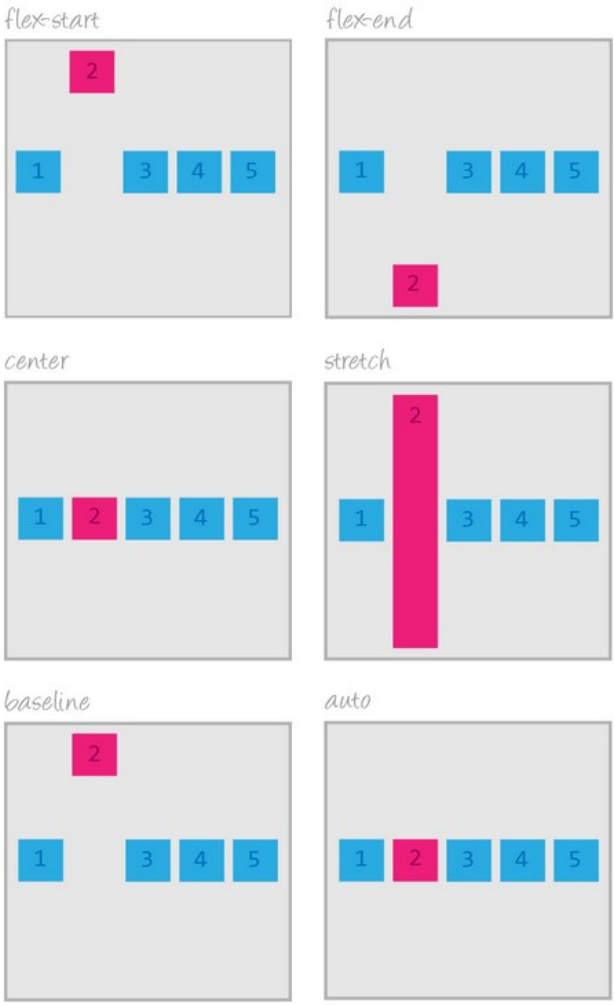
align-content: flex-start | flex-end | center | space-between | space-around | stretch


flex-start
1 2 3 4 5
6 7 8 9 10
11 12
flex-end
1 2 3 4 5
6 7 8 9 10
11 12
center
1 2 3 4 5
6 7 8 9 10
11 12
stretch
1 2 3 4 5
6 7 8 9 10
11 12
space-around
1 2 3 4 5
6 7 8 9 10
11 12
space-between
1 2 3 4 5
6 7 8 9 10
11 12

align-content actúa sobre cada una de las líneas de un contenedor multilínea (no tiene efecto sobre contenedores de una sola línea) y align-items lo hace sobre la línea actual.

align-self actúa igual que align-items pero sobre un ítem específico y no sobre un elemento contenedor. Nos permite cambiar el comportamiento de align-items y sobreescribirlo para ítems concretos que no queremos que se comporten igual que el resto.

```
align-self: auto | flex-start | flex-end | center | baseline | stretch
```



Si se especifica el valor **auto** a la propiedad align-self, el navegador le asigna el valor de la propiedad align-items del contenedor padre, y en caso de no existir, el valor por defecto: **stretch**.

Las siguientes propiedades se aplican sobre los ítems :

Propiedad	Valor	Descripción
flex-grow:	**0** | *[factor de crecimiento]*	Número que indica el crecimiento del ítem respecto al resto.
flex-shrink:	**1** | *[factor de decrecimiento]*	Número que indica el decrecimiento del ítem respecto al resto.
flex-basis:	*[tamaño base]* | **content**	Tamaño base de los ítems antes de aplicar variación.
order:	*[número]*	Número que indica el orden de aparición de los ítems.

flex-grow ==> Indica el factor de crecimiento de los ítems en caso de que no tengan un ancho específico. Si todos tienen el valor 1, por ejempo, todos los ítems tendrán el mismo tamaño, si colocamos un 2 a uno de ellos, entonces éste será mas grande. Si te interesa saber como se calcula visita la página :

https://ed.team/blog/guia-definitiva-de-flexbox-2-flex-basis-flex-frow-flex-shrink

flex-shrink ==> Indica un factor de decrecimiento. Los que tengan un valor numérico mas grande aquí serán mas pequeños.

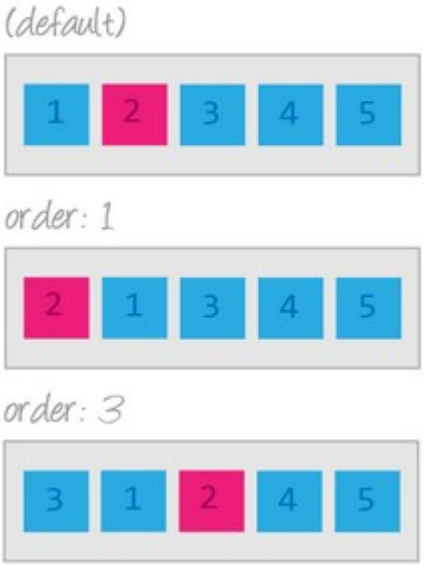
flex-basis ==> Define el taqmaño por defecto que tendrán los ítems antes de aplicarle la distribución de espacio.

Existe una propiedad (flex) que sirve de atajo :

```
/* flex: <flex-grow> <flex-shrink> <flex-basis> */
 flex: 1 3 35%;
```

order ==> Indica el orden en que aparece el ítem según una secuencia numérica.

En el siguiente ejemplo se define la propiedad order para el ítem 2.



Ejemplos :

1) Con la siguiente definición hacemos que un elemento esté centrado horizontal y verticalmente dentro de su contenedor :

```
.container {
    display: flex;
    align-items: center;
    justify-content: center;
}
```

2) Podemos establecer columnas donde todas tienen el mismo height, tengan el contenido que tengan :

```
<header><!-- header stuff --></header>
<div class="container">
    <section class="main"></section>
    <aside class="sidebar"></aside>
</div>
<footer><!-- footer stuff --></footer>
```

```
.container {
    display: flex;
    align-items: stretch;
}
```

3) Navegación flexible.

```
ul.navigation {

    width: 100%;
    display: flex;
    justify-content: space-between;
}
```

El menú de navegación está en horizontal con espacio entre ellos y para un tamaño inferior a 430px estará en columna.

```
@media screen and (max-width: 430px) {

    .navigation {
        flex-direction: column;
        /* other general styles if needed */
    }
}
```

IMPORTANTE !! . Visita esta página para crear flexbox :

https://cvan.io/flexboxin5/

10. Uso de propiedades personalizadas CSS (variables)

Las variables CSS son entidades definidas por autores CSS que contienen valores específicos que se pueden volver a utilizar en un documento. Se establecen mediante la notación de propiedades personalizadas (por ejemplo, --main-color: black;) y se accede mediante la función var() (por ejemplo, color: var (--main-color);).

Declaración de una variable

```
elemento {
  --main-bg-color: blue;
}
```

Utilizando la variable

```
elemento {
  background-color: var(--main-bg-color);
}
```

Ejemplo :

```
:root {

  --main-bg-color: blue;
}

.uno {
  color: white;
  background-color: var(--main-bg-color);
  margin: 10px;
  width: 50px;
  height: 50px;
  display: inline-block;
}


.tres {
  color: white;
  background-color: var(--main-bg-color);
  margin: 10px;
  width: 75px;
}
```

La pseudo-clase `:root` de CSS selecciona el elemento raíz de un árbol que representa el documento. En HTML, `:root` representa el elemento `<html>` y es idéntico al selector `html`, excepto que su especificidad es mayor.

Utilizando var() puede definir múltiples **valores de sustitución (fallback)** cuando la variable dada no está definida aún. Ejemplo :

```
.dos {

  color: var(--my-var, red); /* Rojo (red) si --my-var no esta definida */
}

.tres {
  background-color: var(--my-var, var(--my-background, pink));
/* Rosa (pink) si --my-var y --my-background no están definidas */
}
```